Guia De Desenvolvimento
Rest
Spring MVC
Spring Boot
Git
2026-2
Parte 1

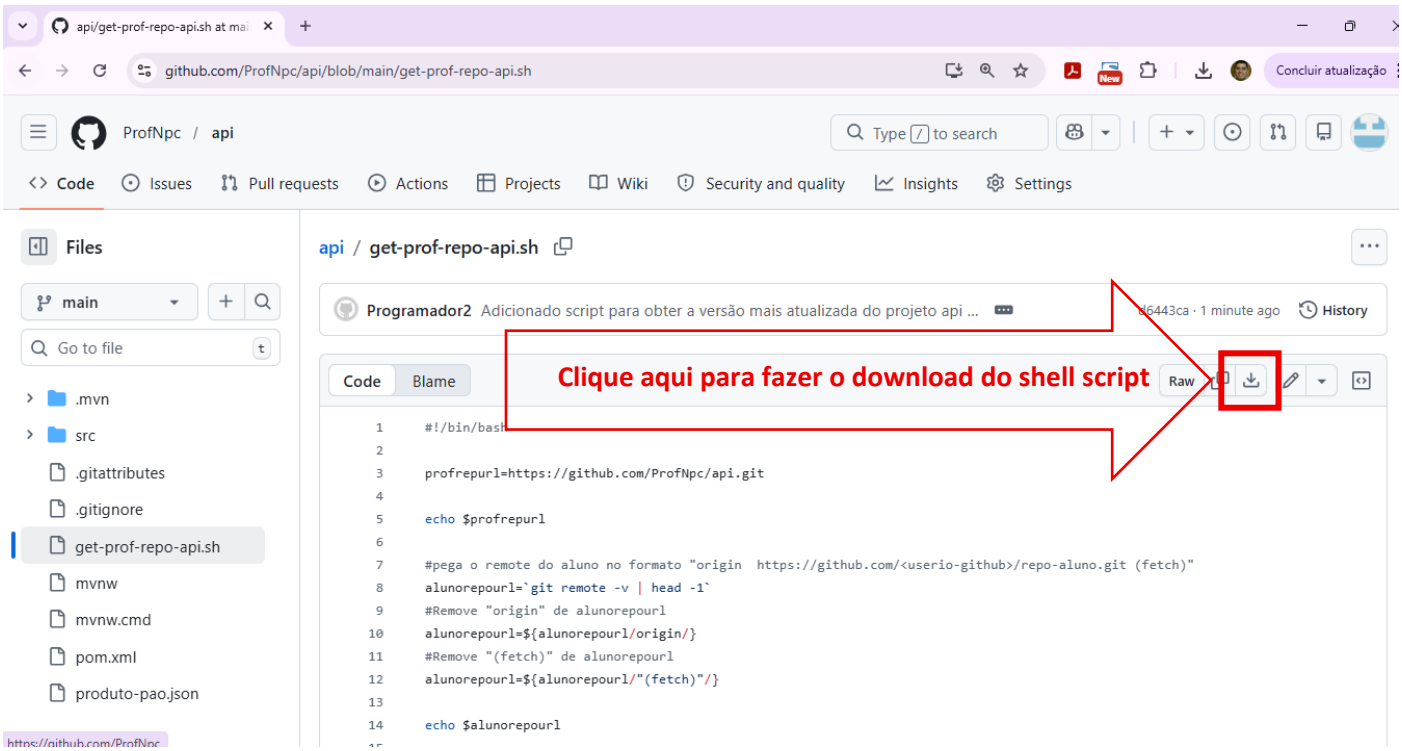
Introdução

Neste documento teremos o passo-a-passo para a construção de uma aplicação web/rest utilizando as ferramentas Spring MVC, Spring Boot e Git.

Este guia se destina a dar suporte as aulas de LIP1 do segundo módulo do curso de informática noturno do ITB Belval, portanto, algumas explicações mais detalhadas serão omitidas pois serão apresentadas pelo professor durante a aula.

O projeto desenvolvido aqui funciona como referência para a criação do *BackEnd* dos projetos de TCC. A cada aula nós avançaremos com o desenvolvimento desse projeto, salvando seu progresso no repositório público https://github.com/ProfNpc/api.

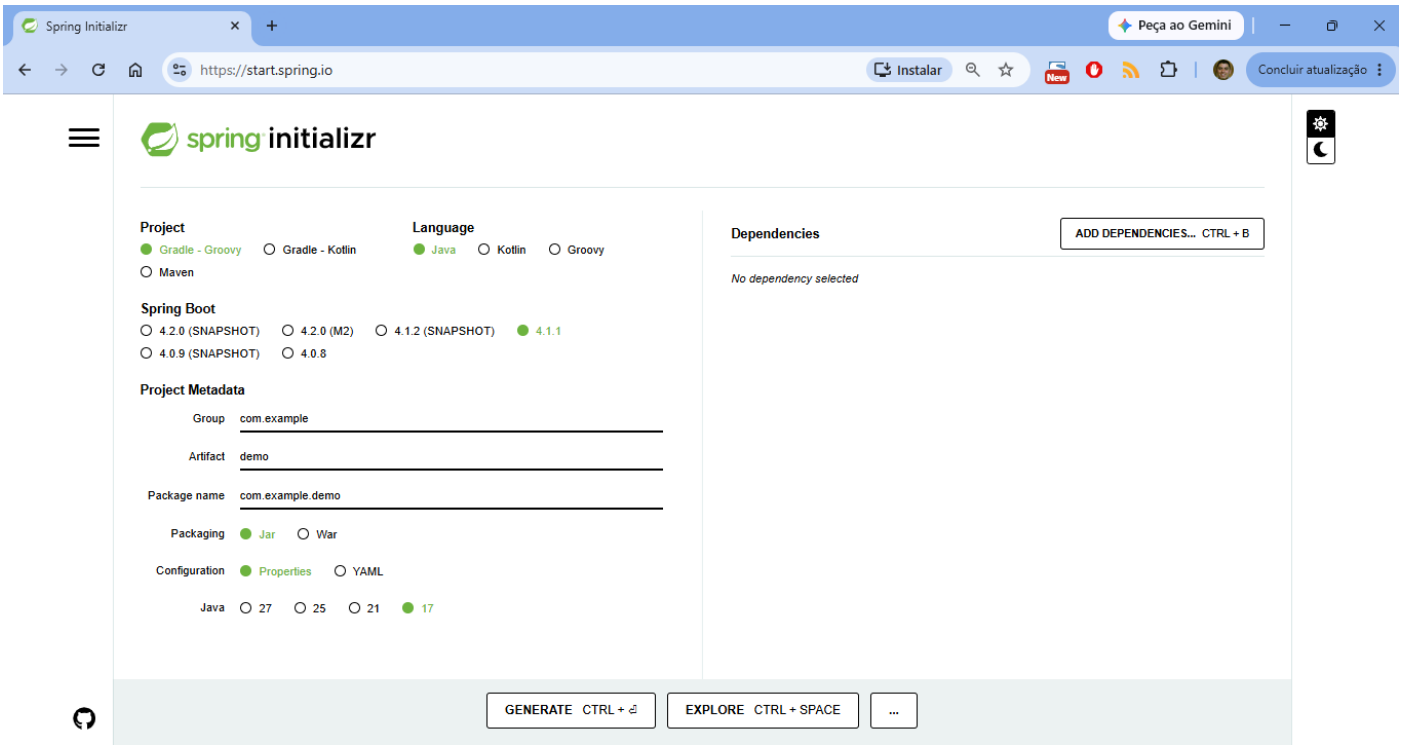
Caso você venha a perder alguma aula, poderá obter a versão mais atualizada feita pelo professor e seguir acompanhando as aulas de onde a turma parou. Para isto basta colocar o *shell script* https://github.com/ProfNpc/api/blob/main/get-prof-repo-api.sh na raiz do projeto api e executá-lo em seguida.



Parte 1

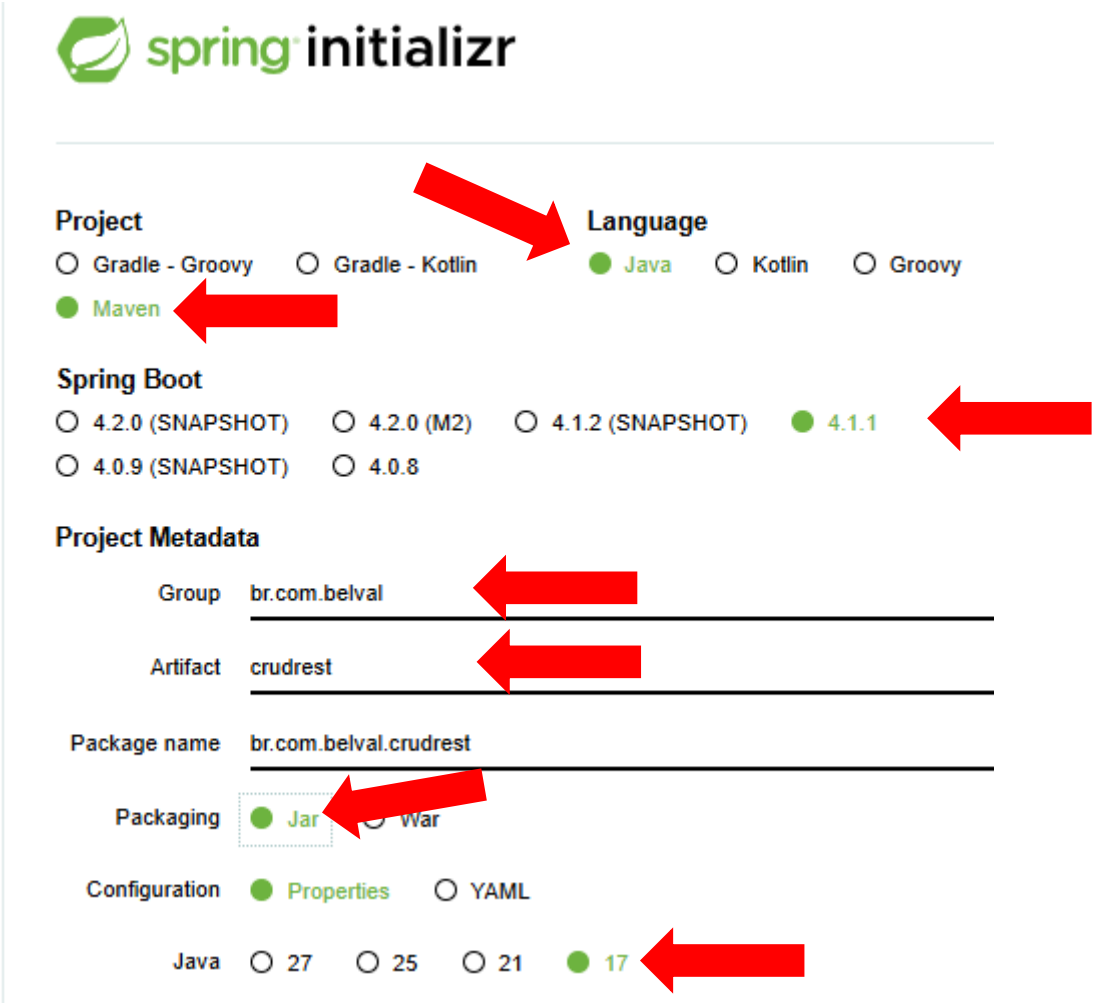
Criar novo projeto

1 – Acesse a página https://start.spring.io/

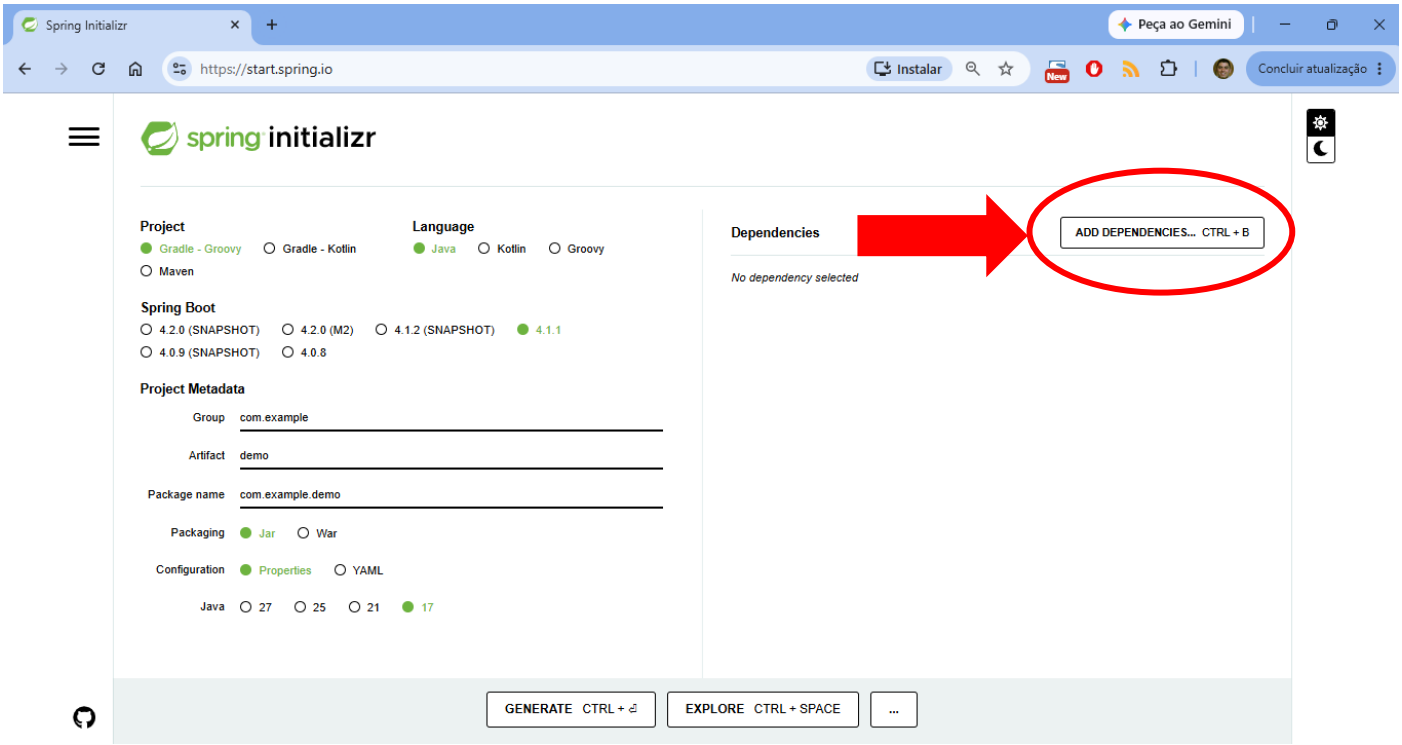


2 – Nessa página preencha da seguinte forma:

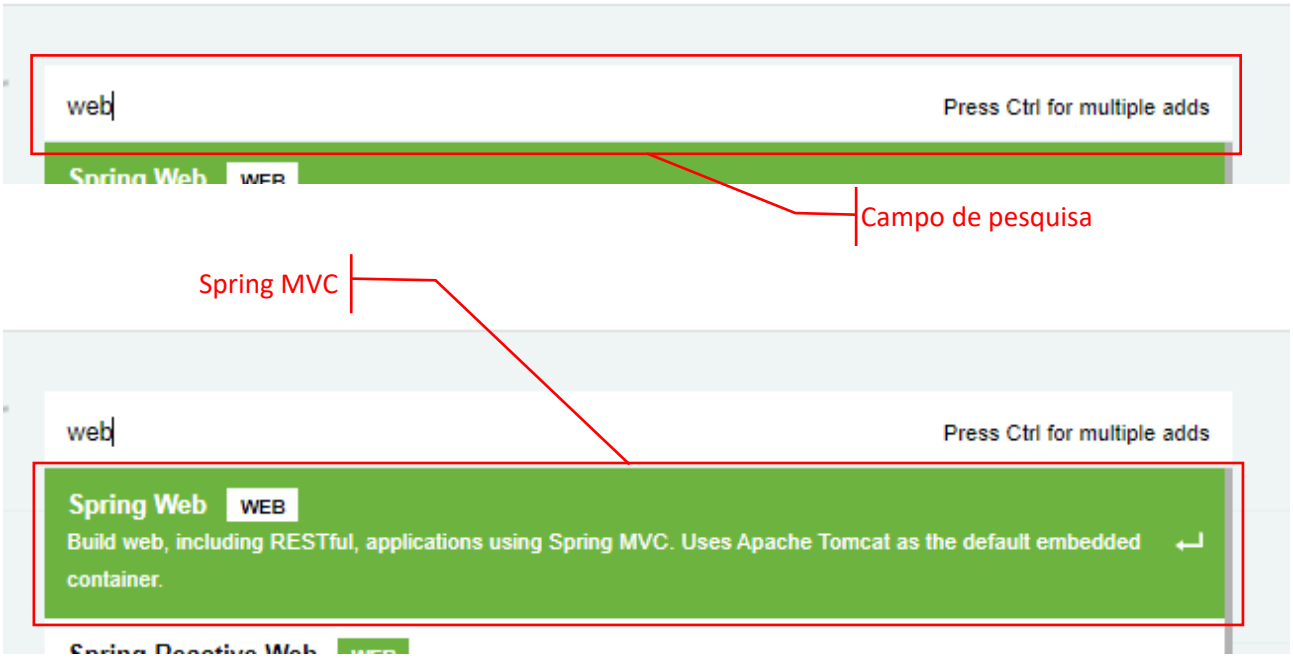
- No campo “Project” selecione a opção “Maven”;
- No campo “Language” deixe “Java” selecionado;
- No campo “Spring Boot” podemos deixar a versão que aparece selecionada, normalmente é a versão estável mais recente. As versões ainda em desenvolvimento, ainda não lançadas oficialmente aparecem com o ter “SNAPSHOT” na frente, NÃO selecione nenhuma versão SNAPSHOT;
- No campo “Group” preencha com “br.com.belval”;
- No campo “Artifact” preencha com “crudrest”;
- No campo “Package name”, após preencher “Group” e “Artifact”, será então apresentado “br.com.belval.crudrest”, deixe dessa forma;
- No campo “Packaging” deixe a opção “Jar” selecionada;
- No campo “Configuration” deixe a opção “Properties”;
- Marque a opção “17” para a versão do java;

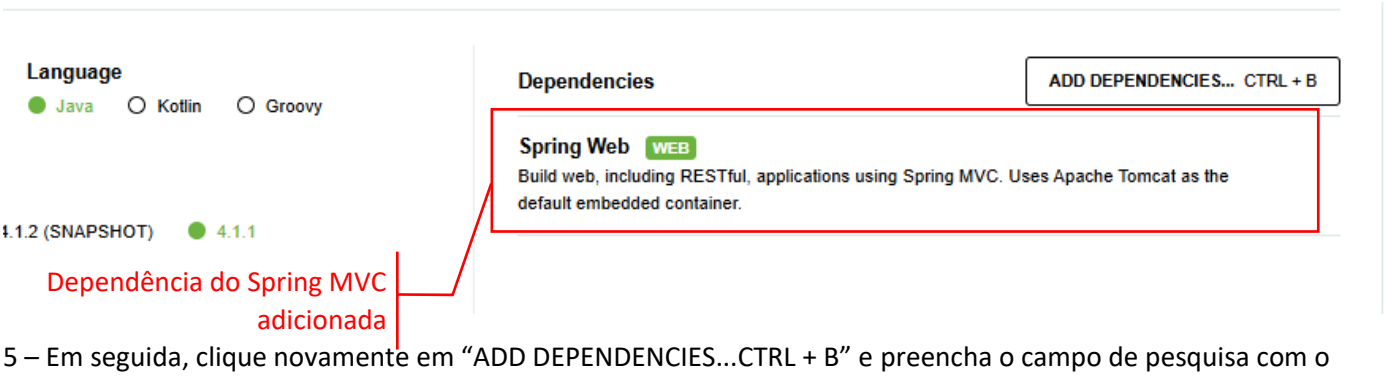


3 – Clique no botão “ADD DEPENDENCIES... CTRL + B”

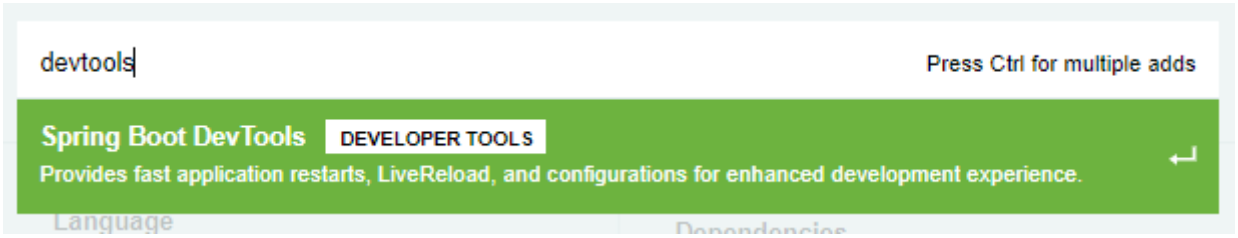


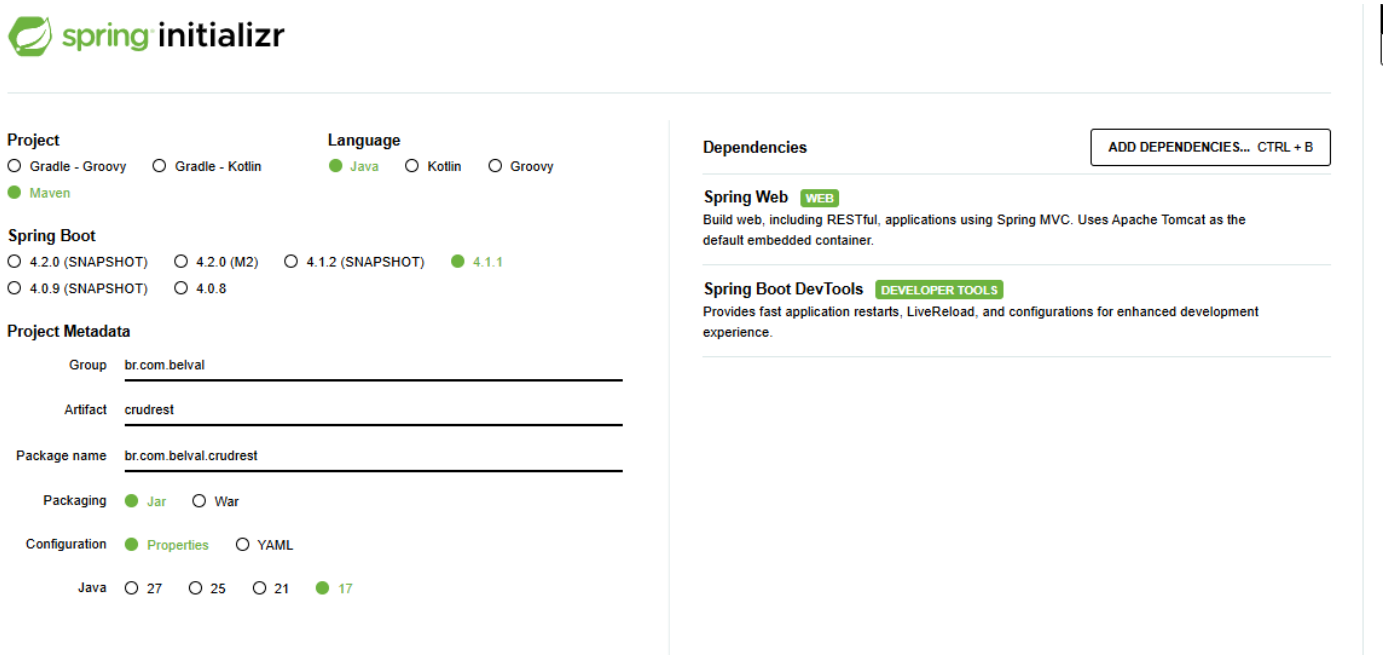
4 – Na próxima tela, preencha o campo de pesquisa com o termo “web”. Provavelmente surgirá uma opção “Spring Web” na primeira opção da lista. Clique nesse item para adicionar o framework Spring MVC ao projeto.



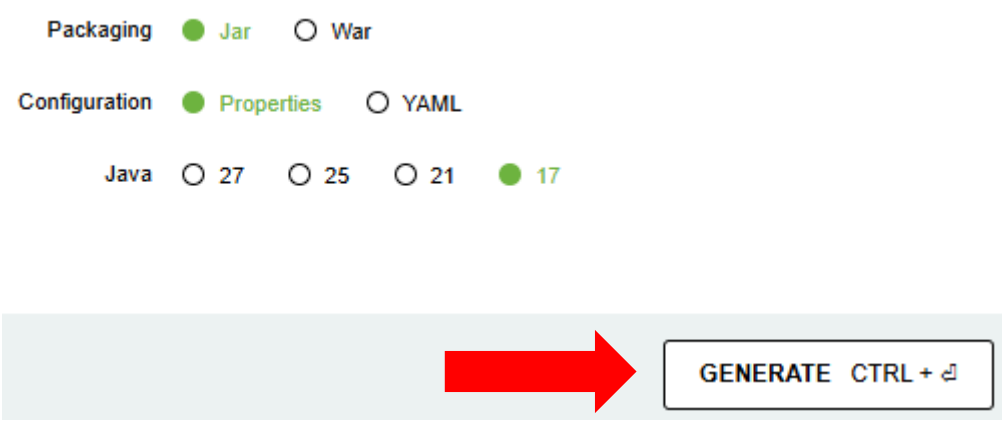


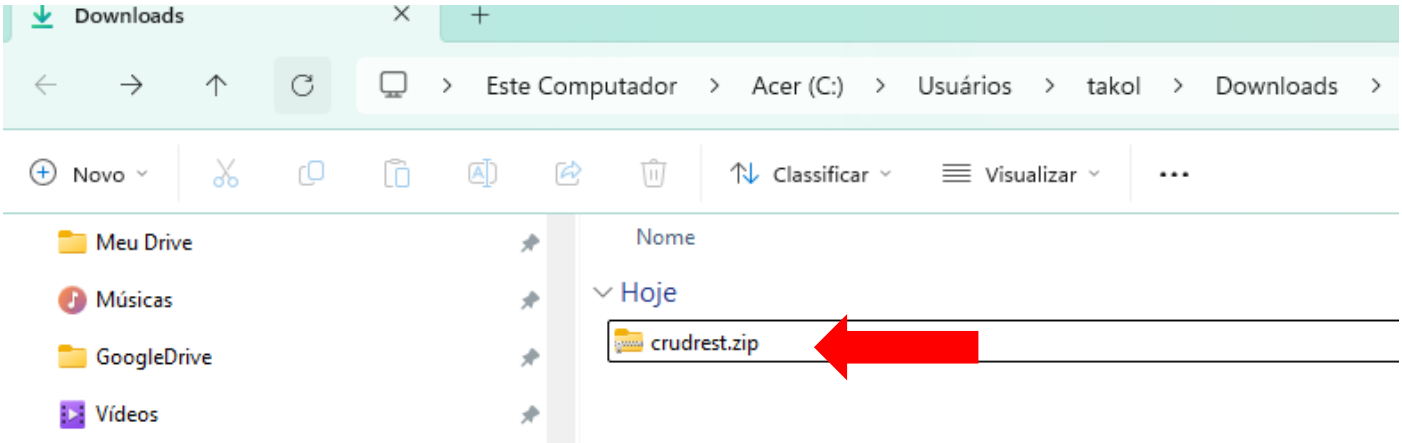
5 – Em seguida, clique novamente em “ADD DEPENDENCIES...CTRL + B” e preencha o campo de pesquisa com o termo “devtools”. Provavelmente surgirá uma opção “Spring Boot DevTools” como única opção da lista. Clique nesse item para adicionar essa ferramenta de produtividade.



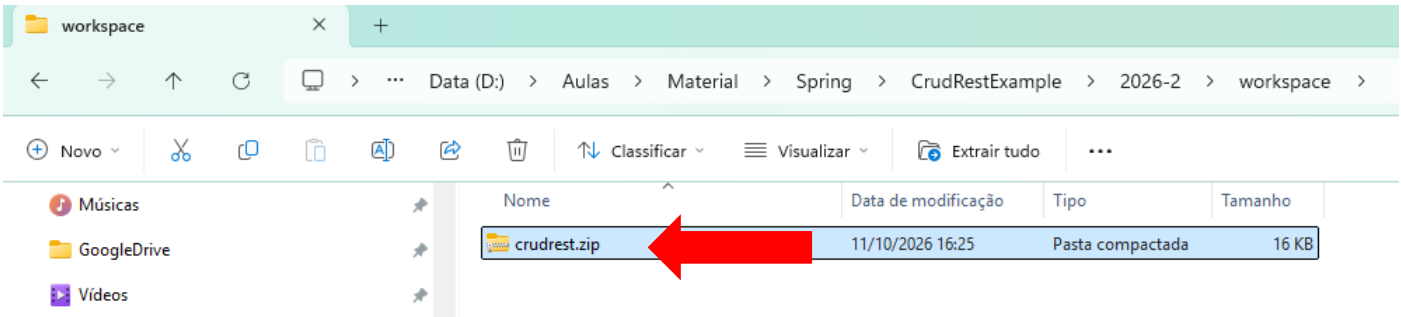


6 – Clique em “Generate CTRL + <ENTER>” para baixar o projeto.

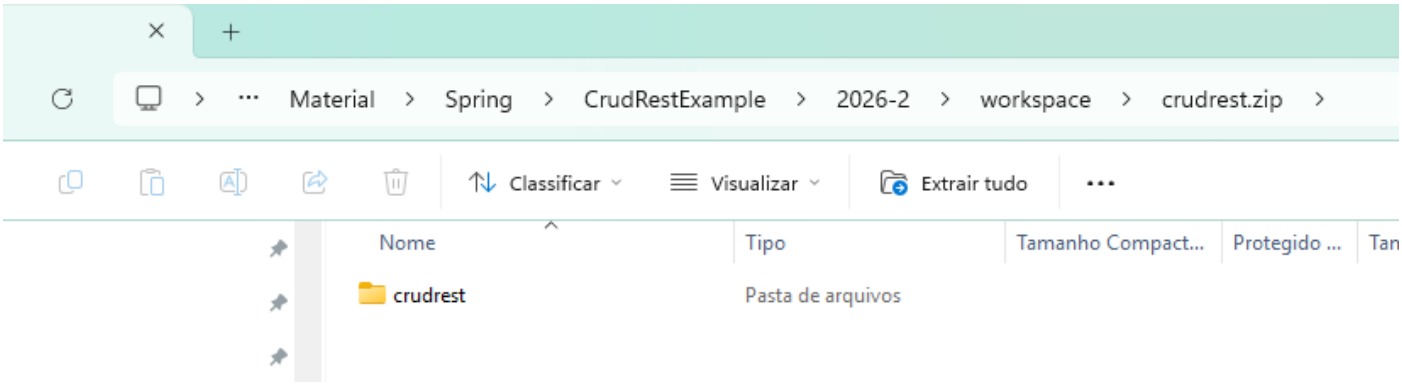




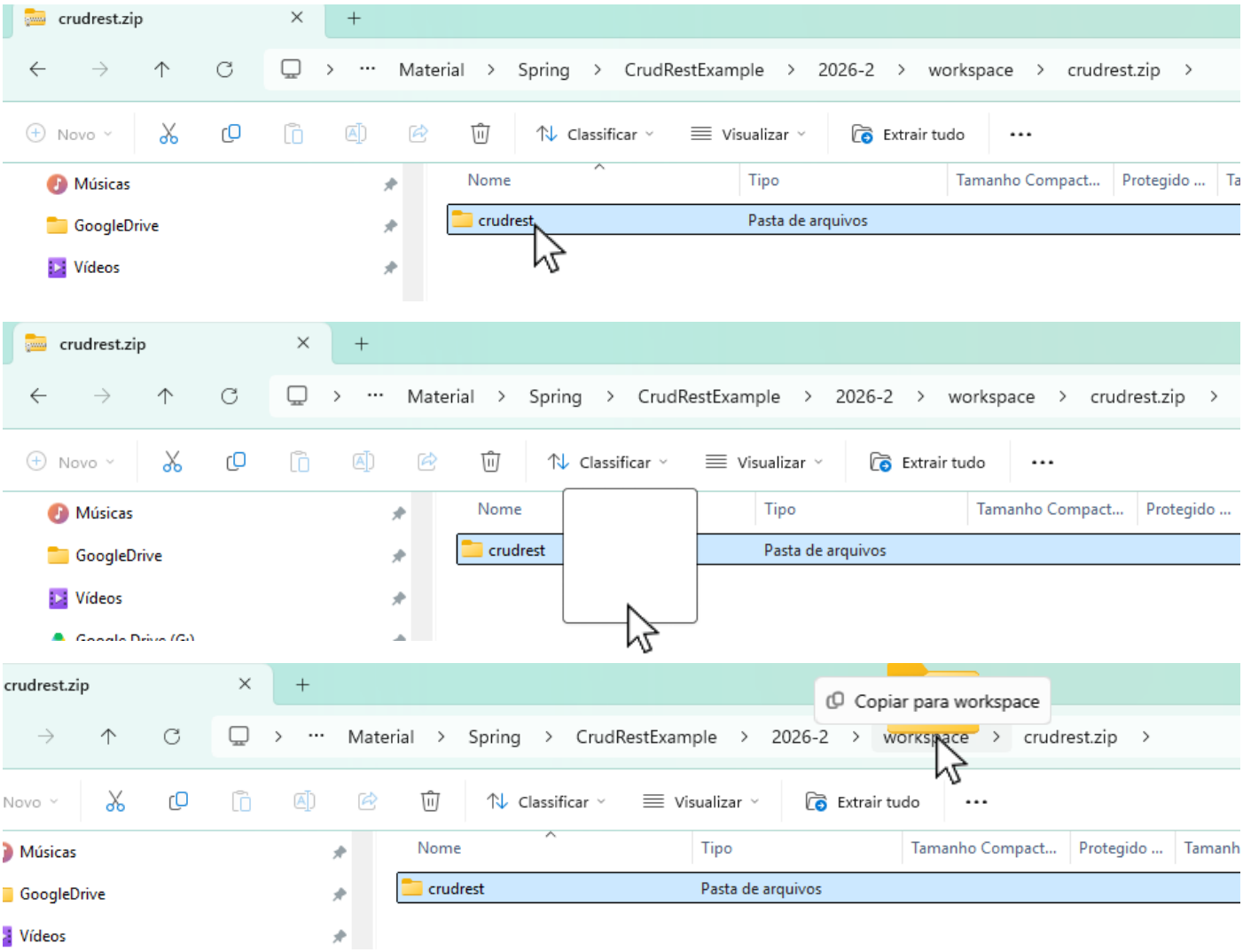
7 – Copie o *.zip baixado para sua pasta “.../workspace”



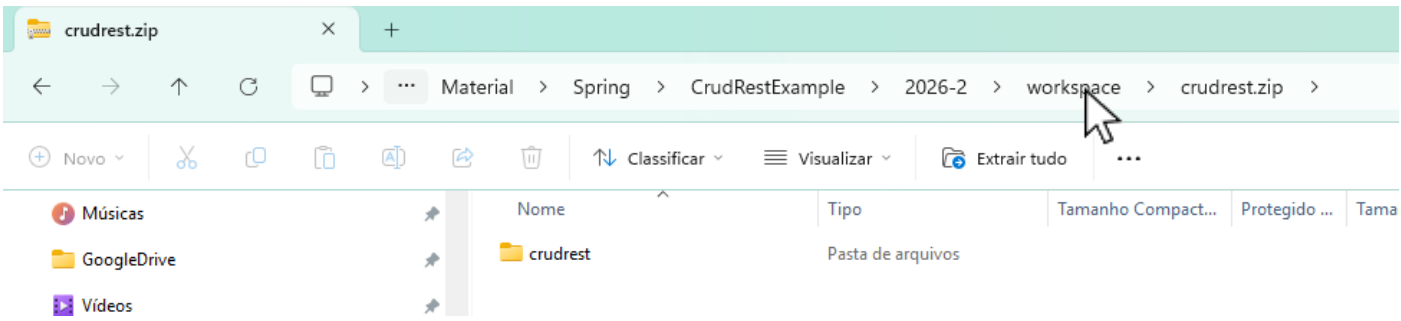
8 – Dê um duplo clique no arquivo crudrest.zip para ver seu conteúdo



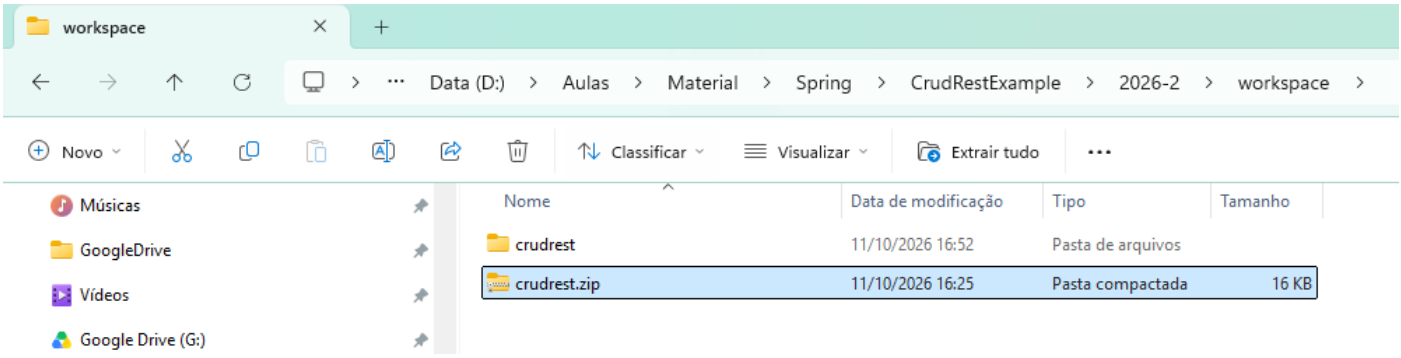
9 – Clique no diretório “crudrest” que está dentro de crudrest.zip e arraste-o pra o diretório workspace da barra de caminho acima



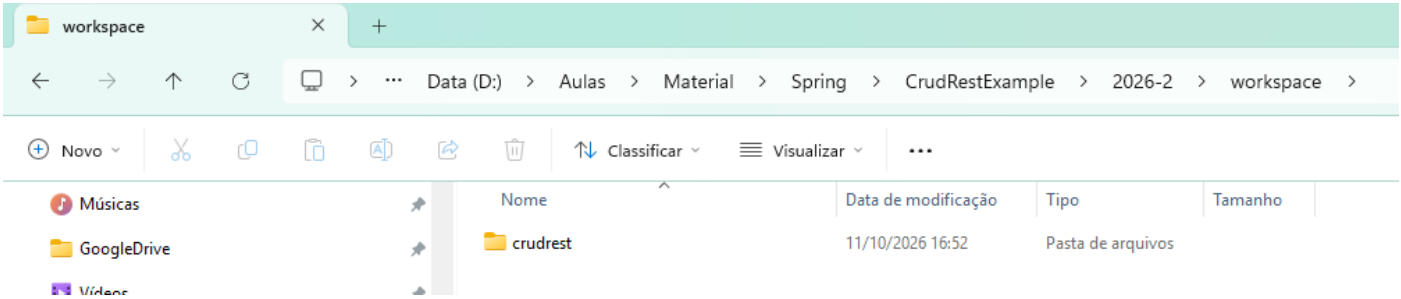
10 – Clique em workspace para sair de dentro do *.zip

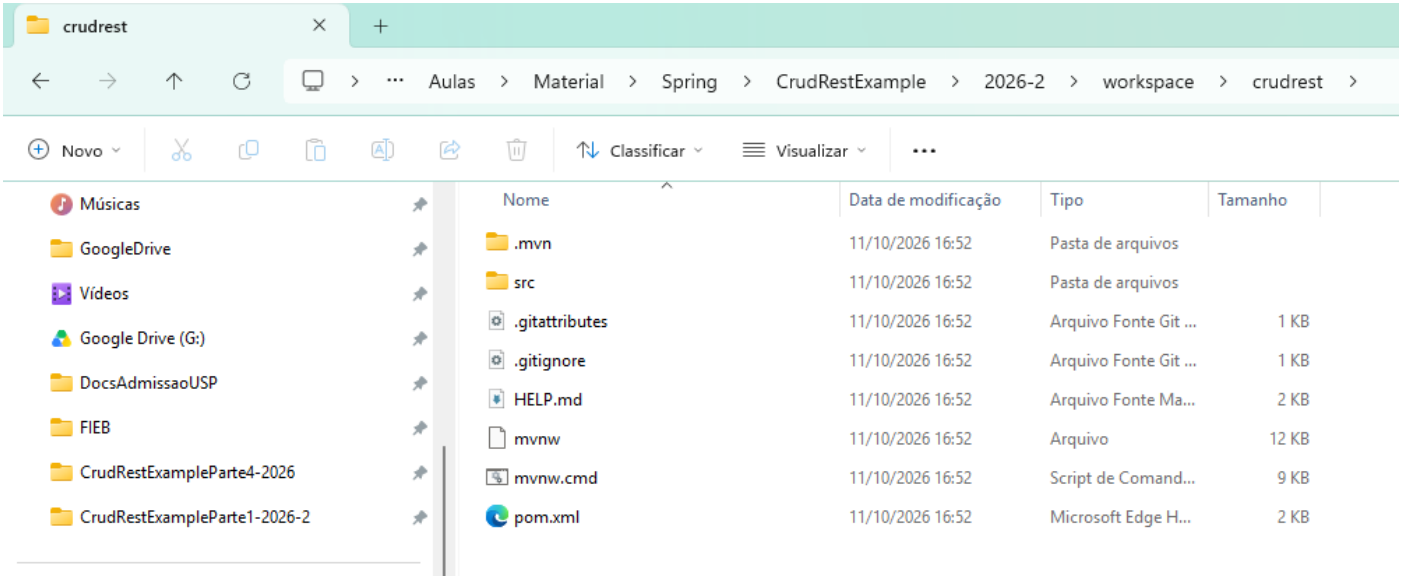


Você verá uma pasta “crudrest” (sem zip) junto do arquivo crudrest.zip



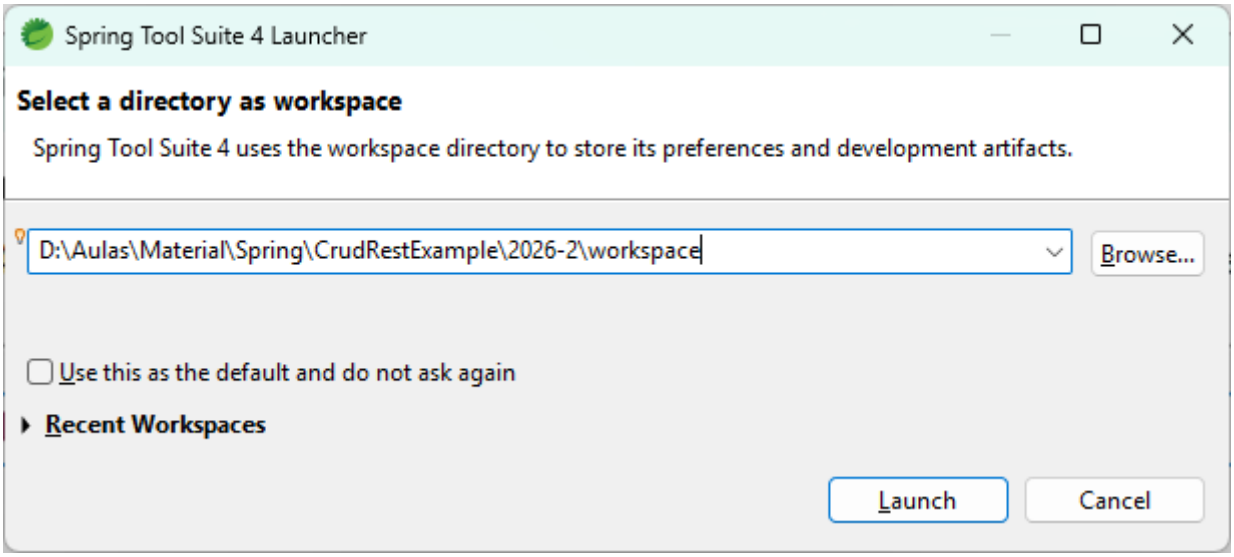
11 – Agora, apague o arquivo crudrest.zip ficando apenas com o diretório crudrest dentro de workspace

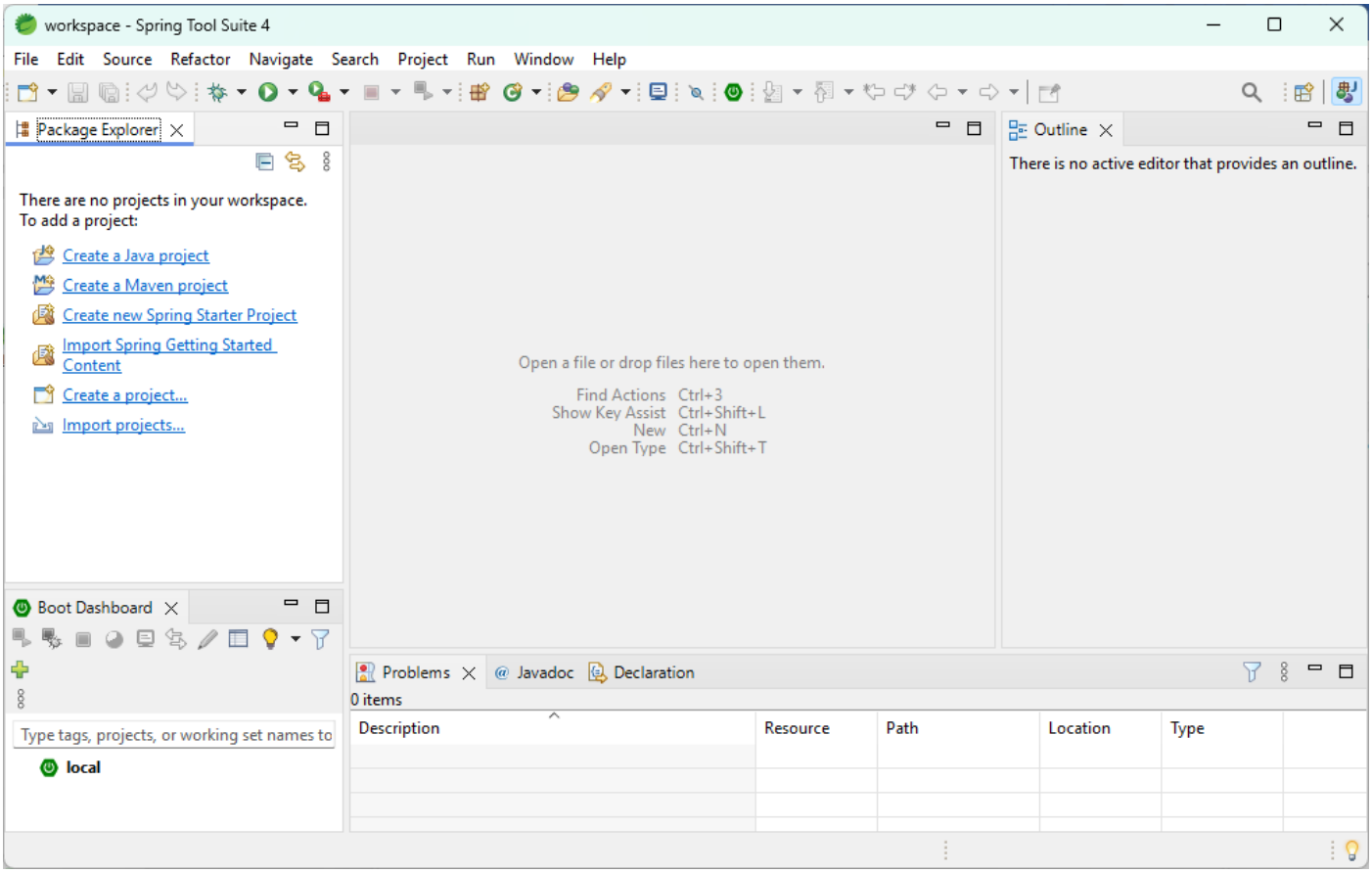




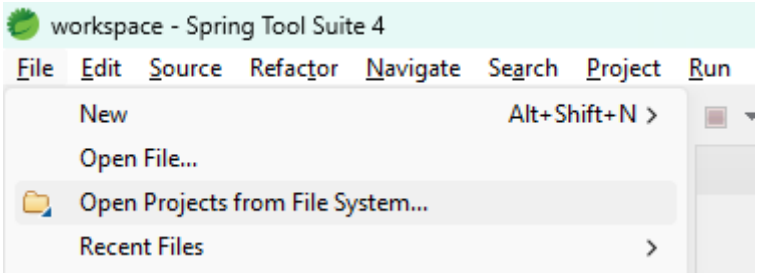
Abrir projeto no Spring Tool Suite

1 – Abra o Spring Tool Suite (STS) clicando no atalho na área de trabalho, preencha o campo “Workspace” com o caminho de seu diretório workspace e, por fim, clique em “Launch”.

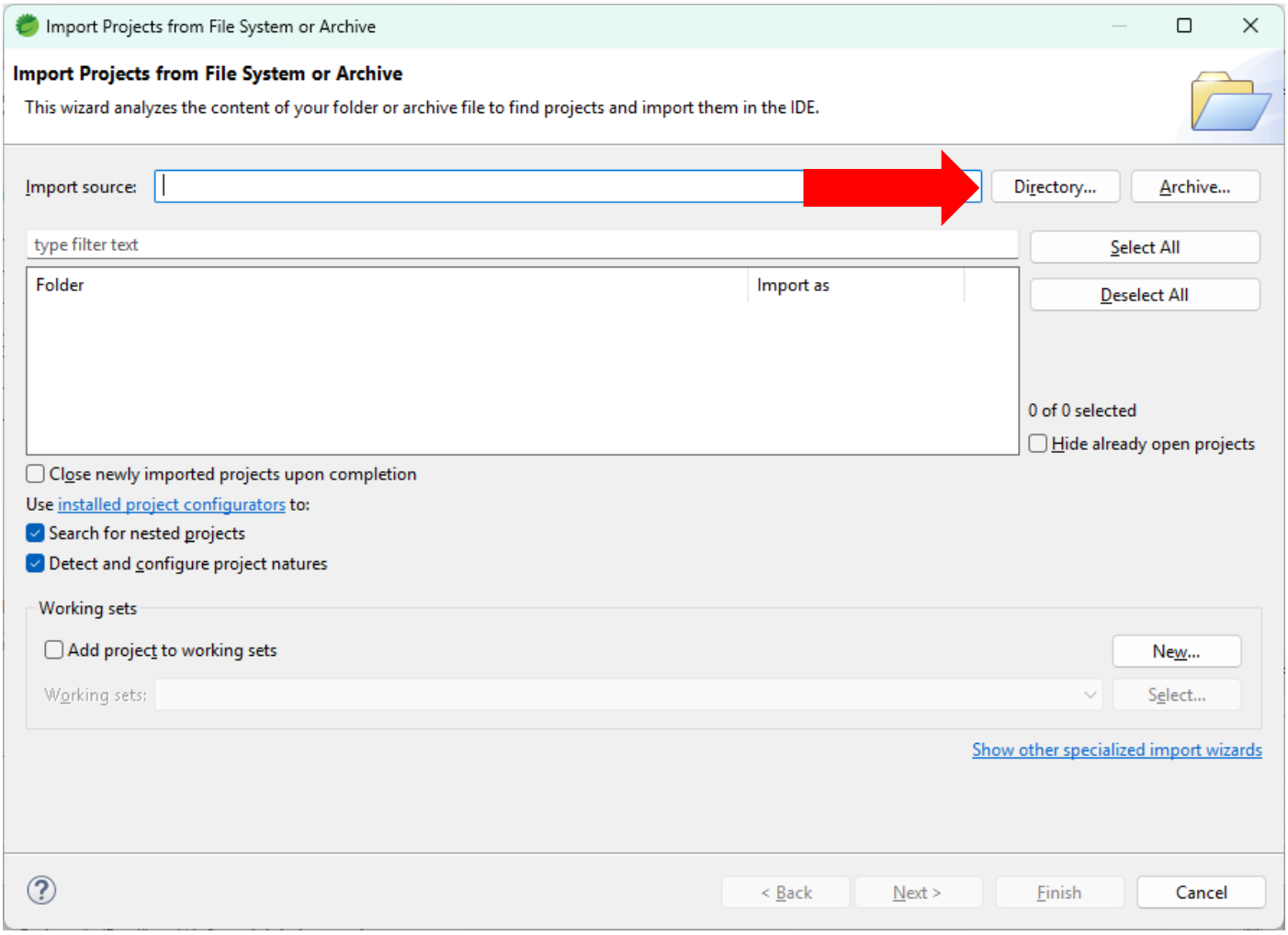




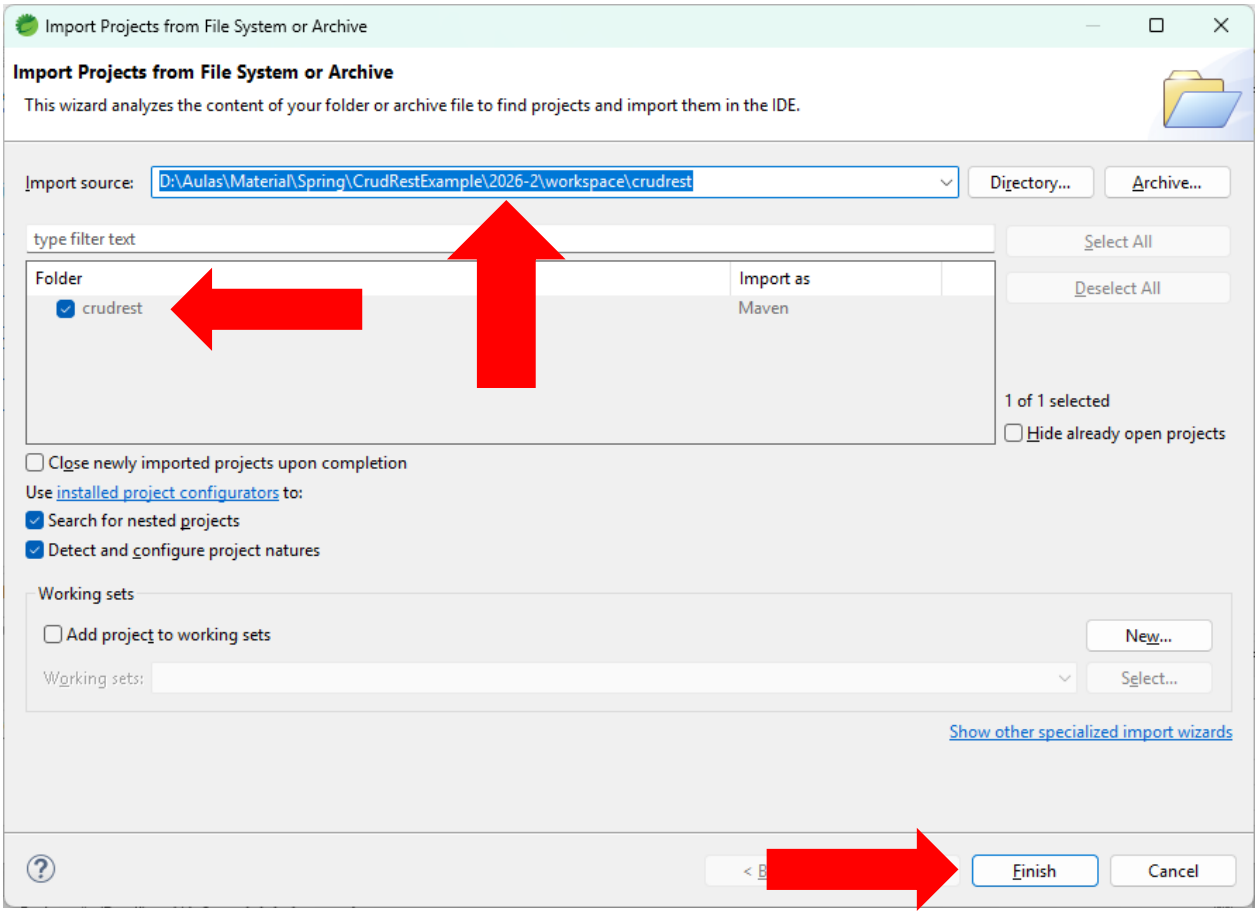
2 – Clique File > Open Projects from File System...



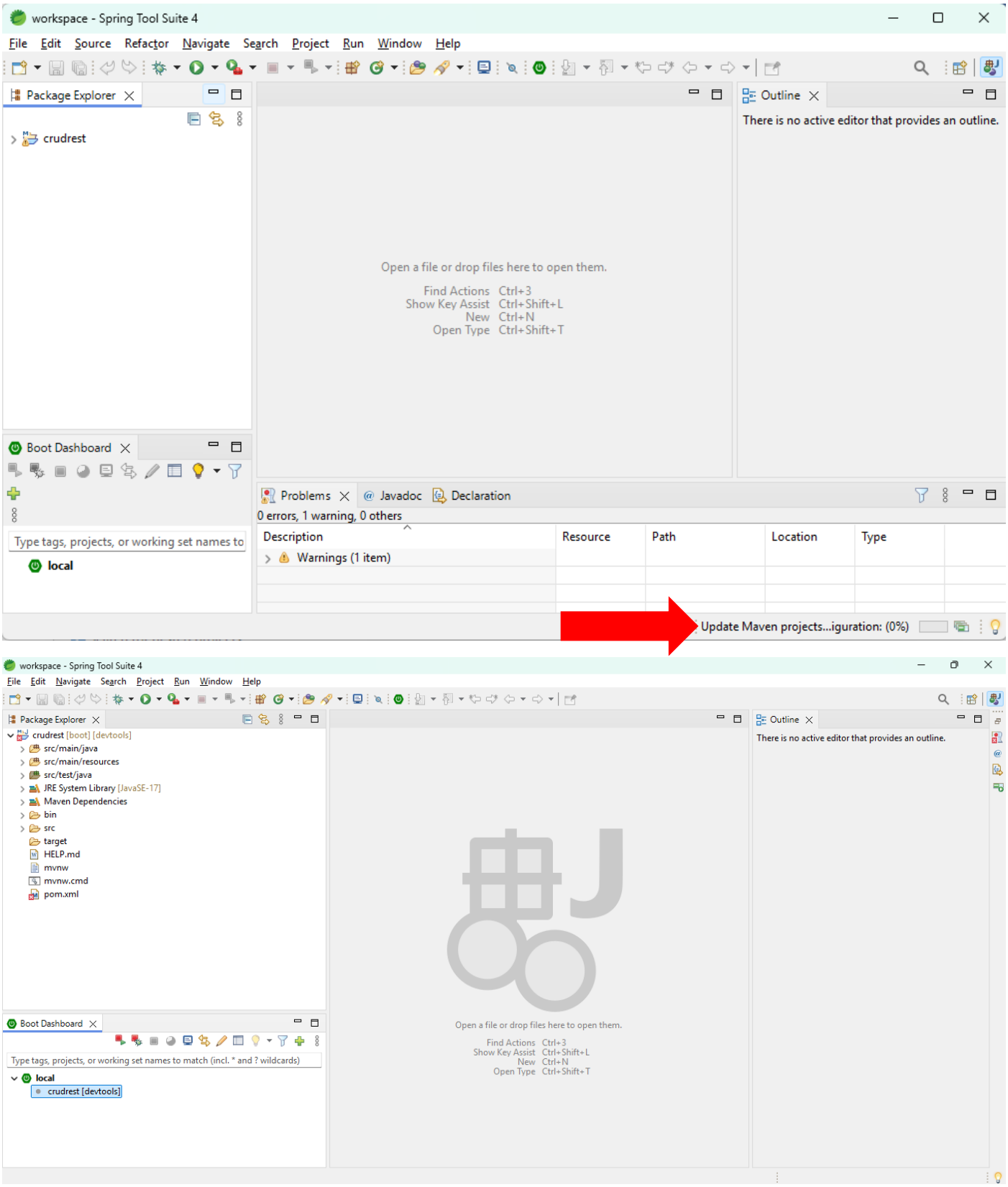
3 – Na tela seguinte clique em “Directory...” e selecione a pasta “.../workspace/crudrest”



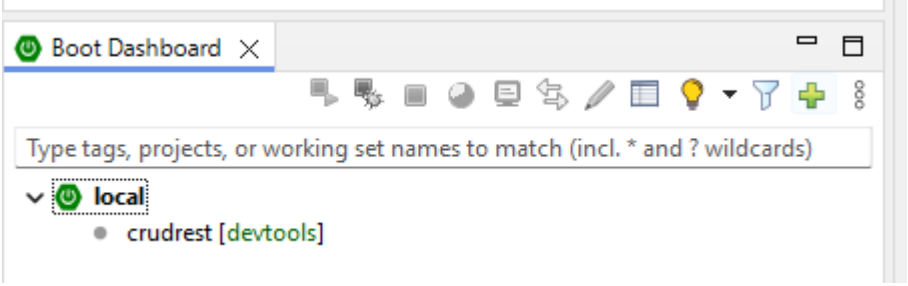
4 – Uma vez que o projeto “crudrest” esteja selecionado, clique em “Finish”.



Aguarde o Maven (gerenciador de dependências) buscar as dependências do projeto e o projeto deverá aparecer como a figura abaixo, sem nenhuma indicação de erro.

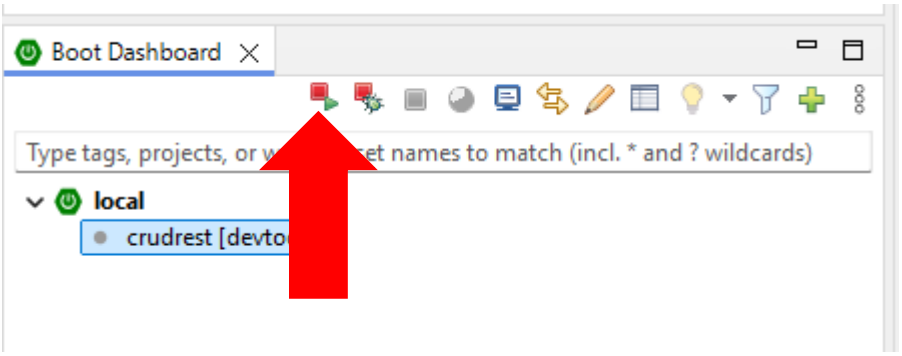


Observe que, caso a importação das dependências tenha sido bem sucedida, nosso projeto crudrest, deverá aparecer “dentro” de “local” na View “Boot Dashboard”.

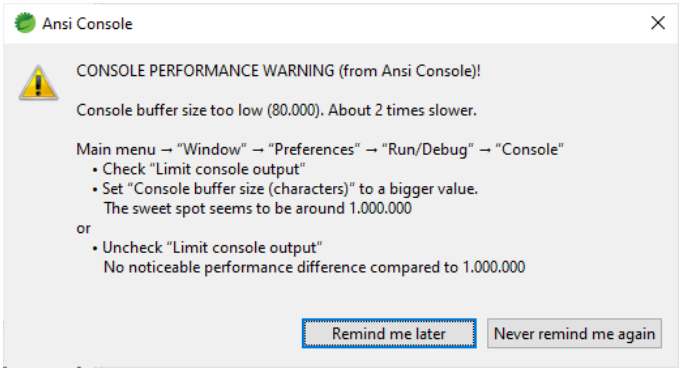


Executar o projeto

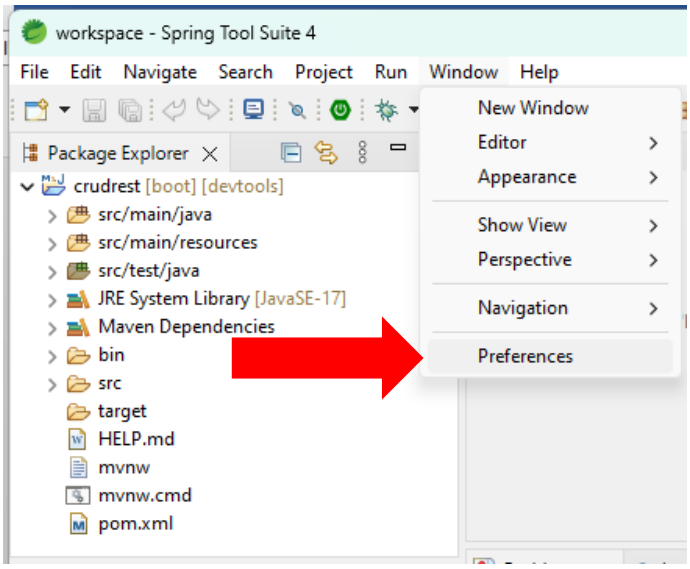
1 – Uma das formas para executar um projeto baseado no Spring Boot é, através da IDE STS, na View “Boot Dashboard”, selecionar o projeto e clicar no botão “Start or Restart”.

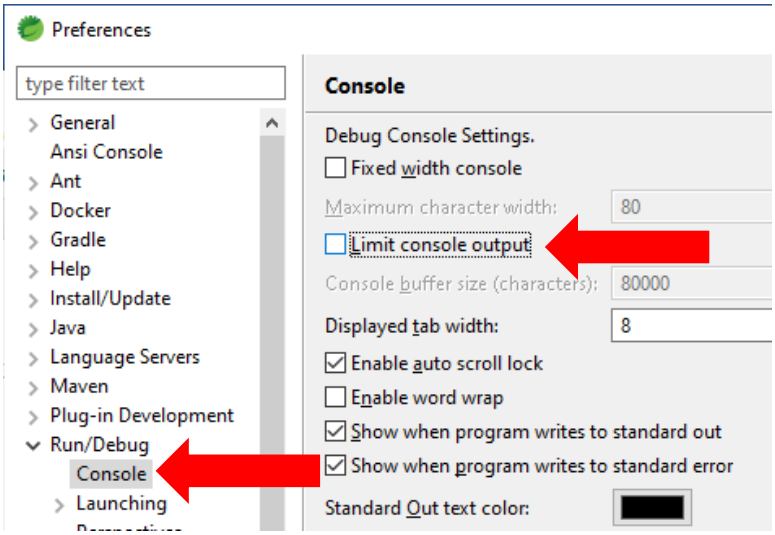


2 - Na primeira vez que você tentar executar seu projeto pode aparecer a tela a seguir, que alerta sobre problemas de performance em função do espaço reduzido do console. Clique em “Never remind me again”.

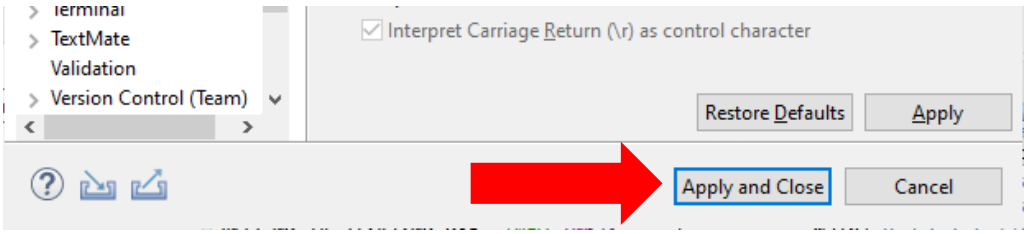


3 – Caso seu ambiente esteja muito lento você pode seguir a recomendação dessa tela e ir no menu Window > Preferences > Run/Debug > Console e desmarcar a opção “Limit console output”.

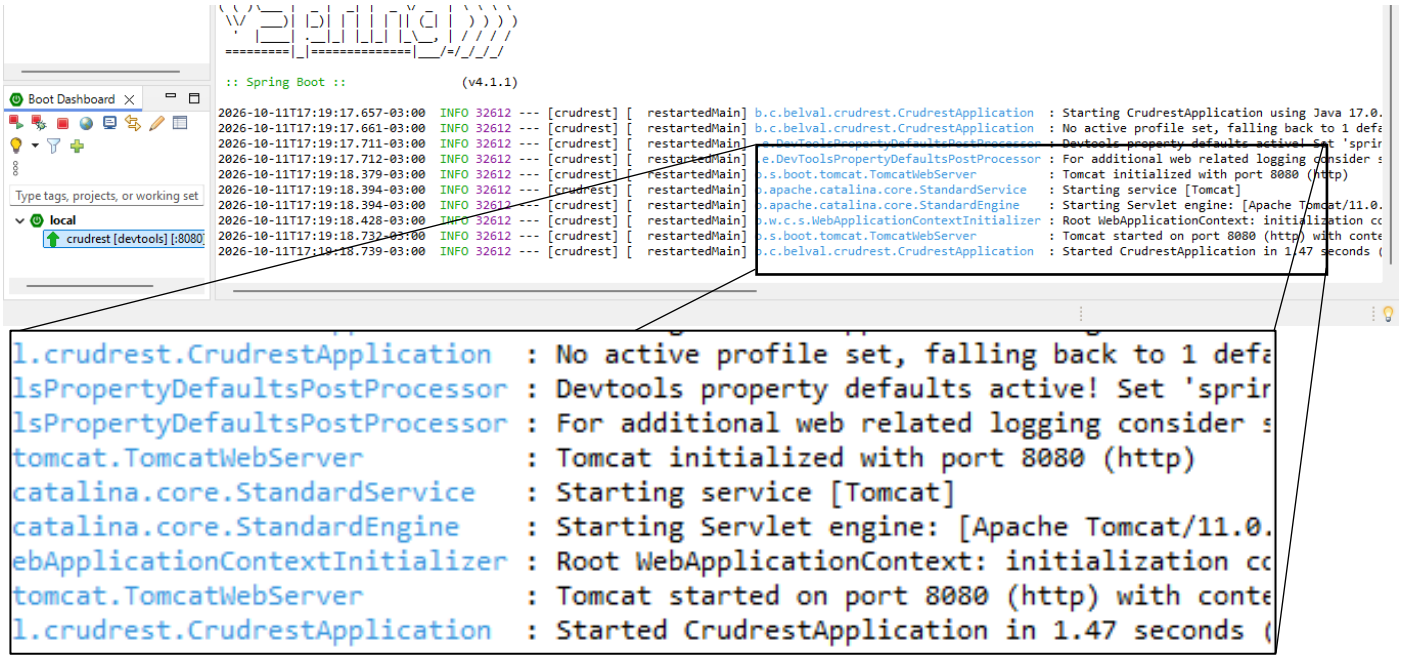




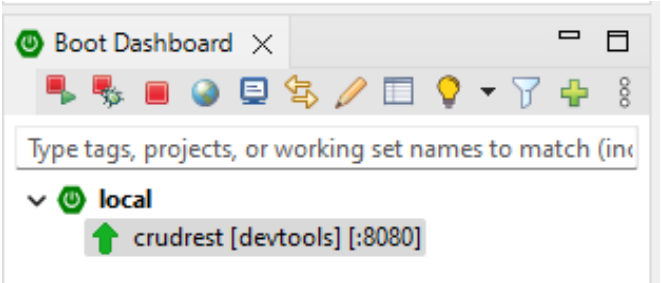
4 – Então é só clicar em “Apply and Close”



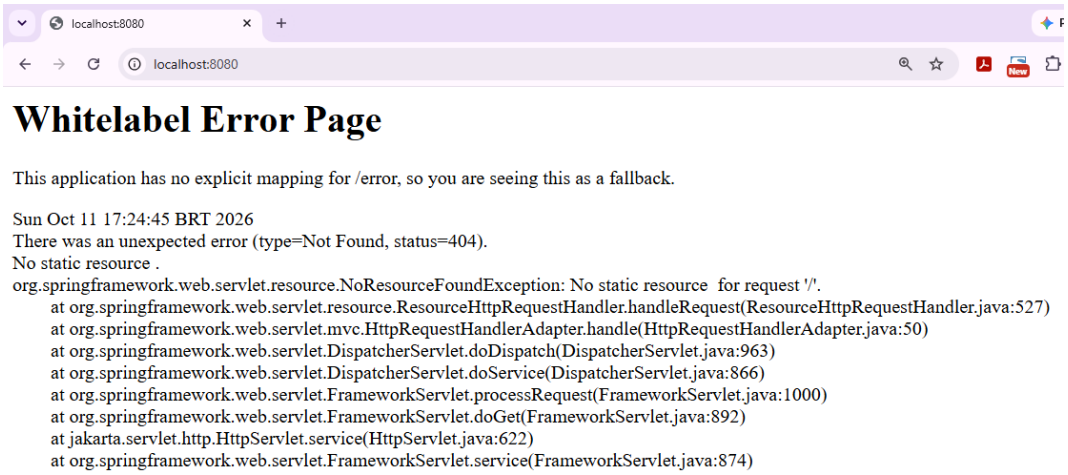
Após executar o projeto, aparecerá, no console, o registro dessa execução com algumas informações, entre elas, que o servidor web tomcat foi executado e, com ele, nossa aplicação. O tomcat estará “ouvindo” a porta 8080 da máquina. É através dessa porta que acessaremos nossa aplicação.



No “Boot Dashboard”, aparecerá



5 – Abra o browser e digite na barra de endereço http://localhost:8080



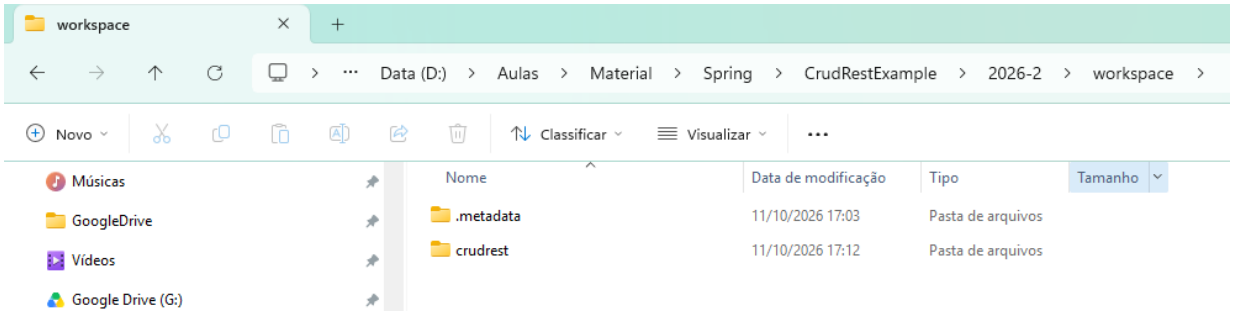
Caso apareça o texto “Whitelabel Error Page”, isso significa que nossa aplicação está funcionando.

Salvando uma fotografia do projeto

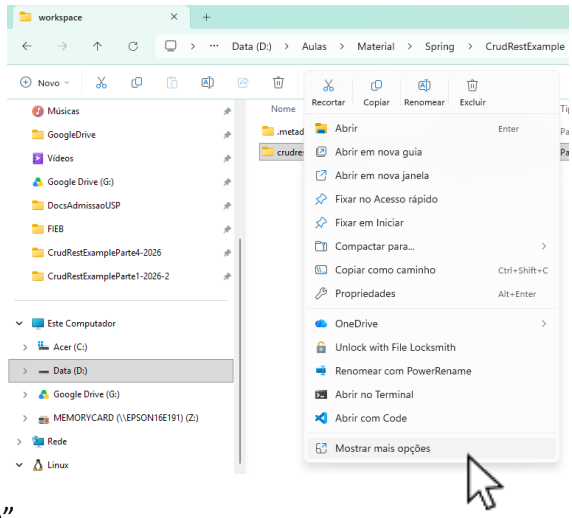
Nesse ponto temos um projeto corretamente configurado e é importante salvarmos uma fotografia desse ponto exato do projeto para que, caso venhamos a “quebrar” nosso projeto ao tentar adicionar uma nova funcionalidade, possamos voltar a ele depois.

Podemos fazer isso através do uso de ferramentas de controle de versão e a ferramenta mais utilizada hoje em dia é o Git. Ele pode ser um pouco complicado às vezes, mas se tratando de uma ferramenta tão utilizada na indústria de desenvolvimento de software é bom começarmos a aprender como utilizá-lo.

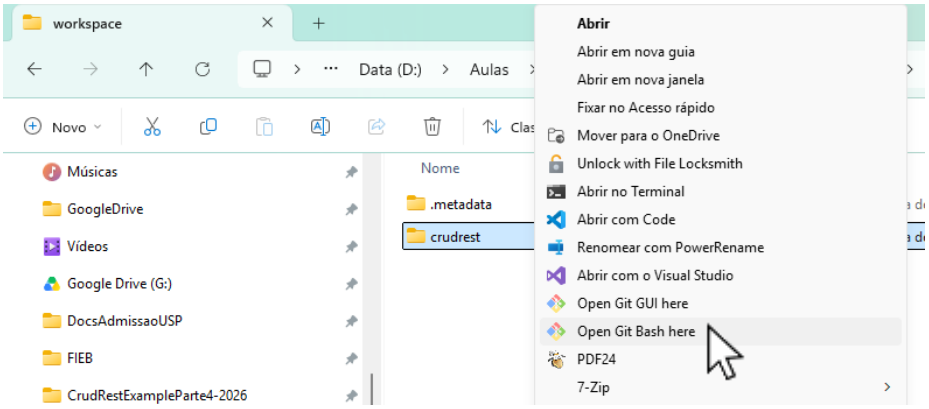
1 – Abra o workspace no Windows Explorer

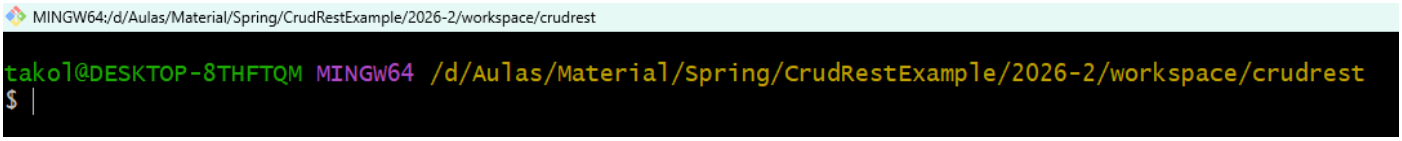


2 – Clique com o botão direito do mouse no diretório crudrest e clique em “Mostrar mais opções”



3 – Clique em “Open Git Bash here”



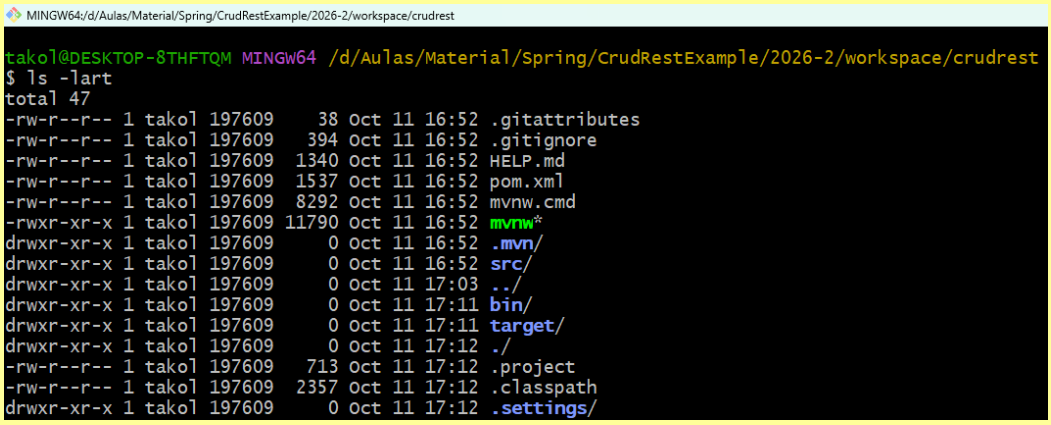


Observação:

O Git Bash simula um terminal bash Linux, isto significa que os seguintes comandos estão disponíveis:

- `cd`
  Permite trocar de diretório.
  Você pode digitar `cd "<caminho do diretório do projeto>"` para ir para o diretório do projeto.
  No Git Bash é possível utilizar um caminho do Windows desde que esse caminho esteja entre aspas.
- `ls`
  Permite listar o conteúdo de um diretório.

  Se você digitar `ls -lart` poderá ver a lista dos arquivos e diretório do diretório corrente com vários detalhes, em "`-lart`" cada letra possui um efeito no resultado obtido



  - `l` (éle minúsculo) – Mostra um arquivo/diretório por linha, mostra outras informações além do nome do arquivo/diretório como o dono do arquivo/diretório, seu grupo, tamanho e data da última modificação. A letra dessa opção vem da palavra *long* justamente porque apresenta uma longa descrição para cada item.
  - `a` – Inclui na lista arquivos/diretórios ocultos (que começam por "`.`"), o diretório corrente ("`.`") e o diretório pai ("`..`"). A letra dessa opção vem da palavra *all*, tudo em inglês, justamente porque mostra todos os itens presentes no diretório.
  - `t` – Ordena a lista de acordo com a data de modificação, do mais recente para o mais antigo. Vem da palavra *time*.
  - `r` – Inverte a ordem, ou seja, junto com a opção `t`, os arquivos com alteração mais recente ficam na parte de baixo da lista, normalmente esses arquivos são os que estamos interessados, afinal, muitas vezes, acabamos de alterá-los para fazer alguma coisa com eles.

Para copiar algo que aparece no terminal do Git Bash podemos utilizar o atalho CTRL + INSERT e para colar algo que foi copiado anteriormente, do próprio Git Bash ou de qualquer outro lugar, podemos utilizar o atalho SHIFT + INSERT.

4 – Execute os seguintes comandos para configurar adequadamente seu diretório como um diretório gerenciado pelo Git.

```
git config --global init.defaultBranch main

git config --global core.autocrlf true

git config --global core.safecrlf warn

git init

git config --local user.name "Seu Nome"

git config --local user.email "seu_email@qualquercoisa.com"
```

Não esqueça de trocar “Seu Nome” pelo seu nome e “seu_email@qualquercoisa.com” pelo seu e-mail, de preferência pelo e-mail que você utiliza no GitHub.

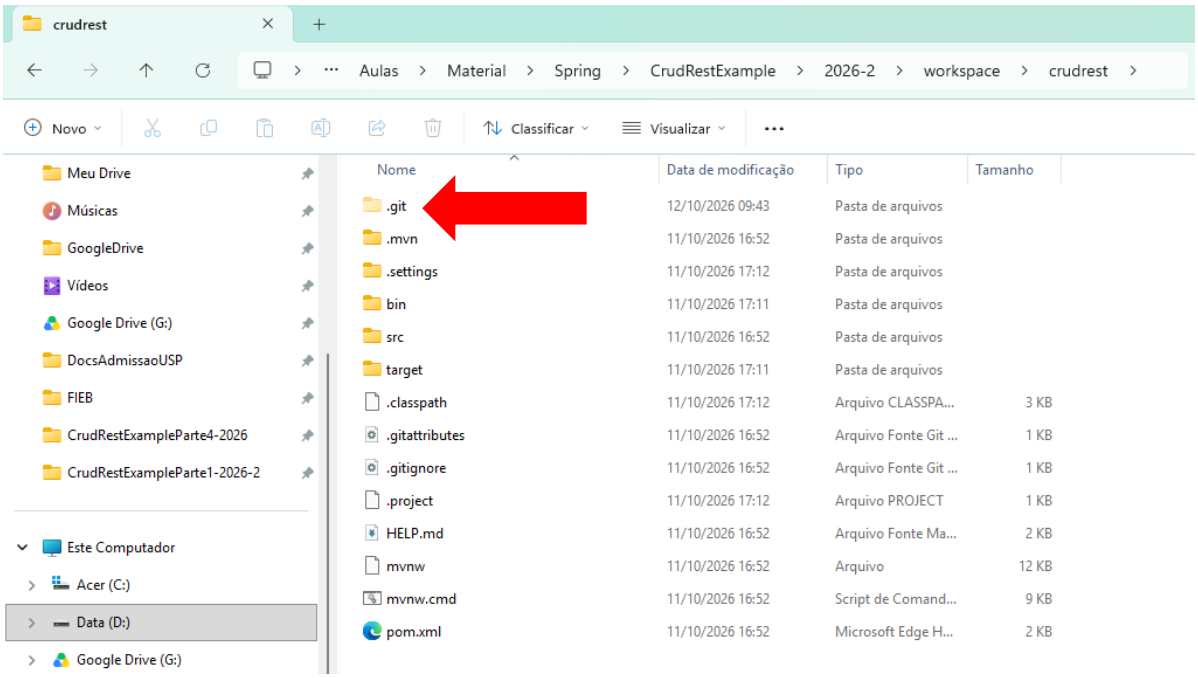
Você pode copiar esses comandos e colar no GitBash com SHIFT + INSERT e, ao pressionar a tecla ENTER, os comandos serão executados na sequência.

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest
$ git config --global init.defaultBranch main
git config --global core.autocrlf true
git config --global core.safecrlf warn
git init
git config --local user.name "ProfNpc"
git config --local user.email "professornpc@gmail.com"
Initialized empty Git repository in D:/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest/.git/

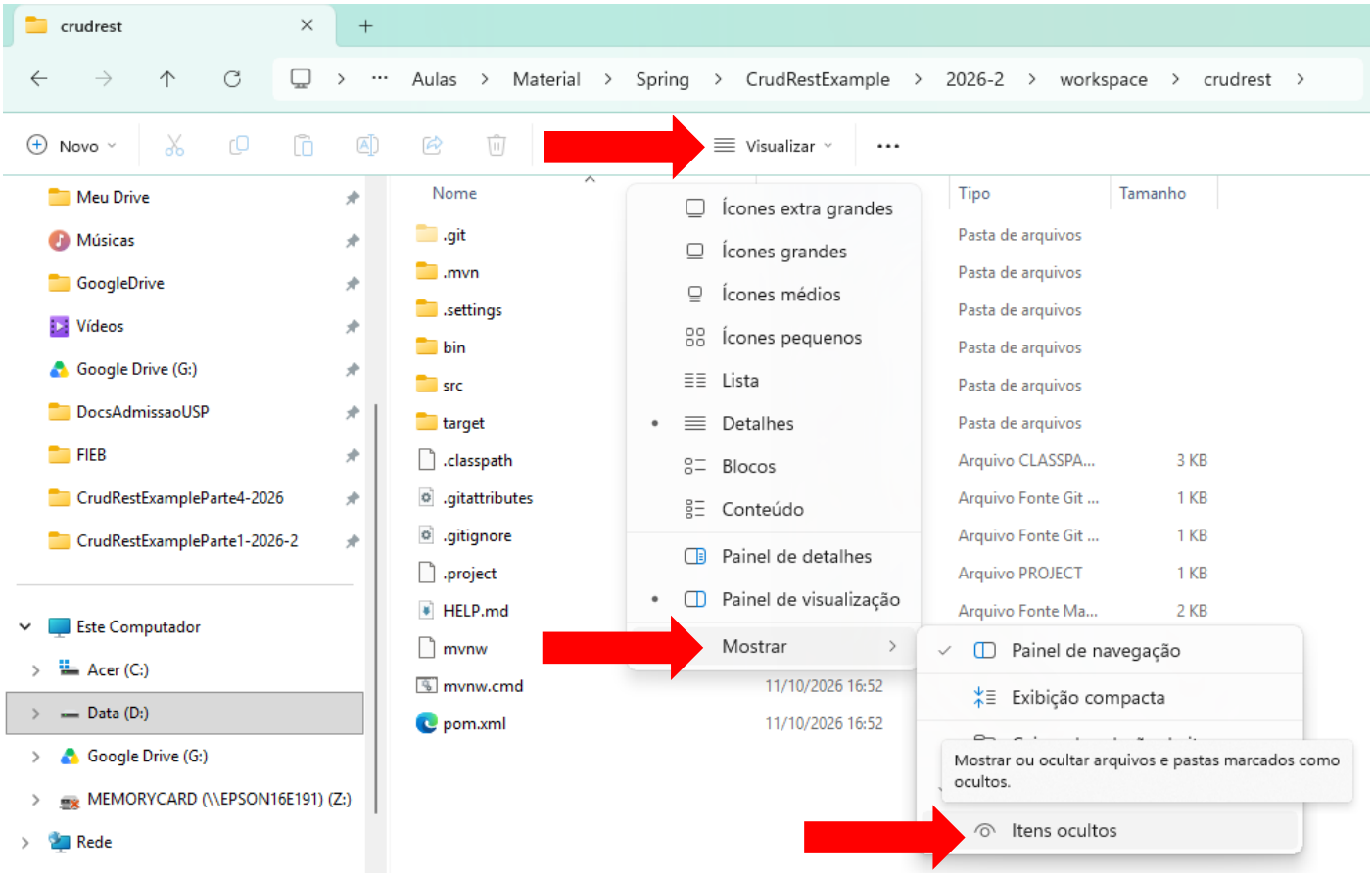
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Os comandos `git config --global core.autocrlf true` e `git config --global core.safecrlf warn` devem ser utilizados apenas se você estiver no Windows, no Linux e no macOS esses comandos são diferentes.

O comando `git init` transforma o diretório corrente em um diretório gerenciado pelo Git e cria um diretório oculto “.git”.

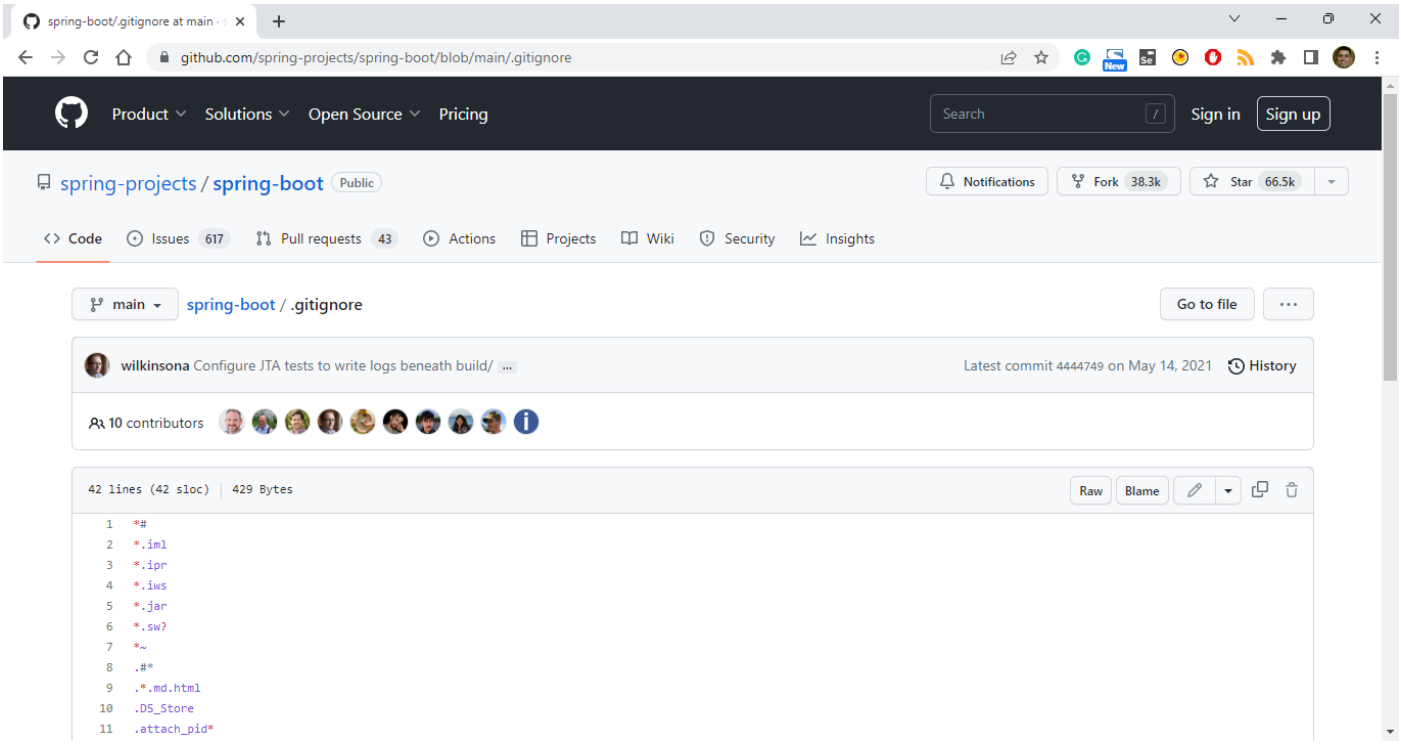


Se você não estiver vendo esse diretório, clique na opção “Visualizar” > “Mostrar” > “Itens Ocultos”

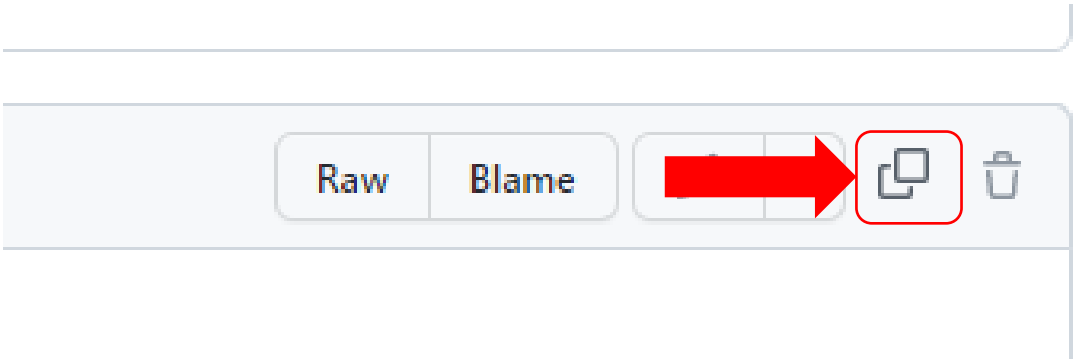


Esse diretório será o repositório que conterá todas as diferentes versões do nosso software nos permitindo recuperar versões antigas caso necessário. NÃO APAGUE ESSE DIRETÓRIO.

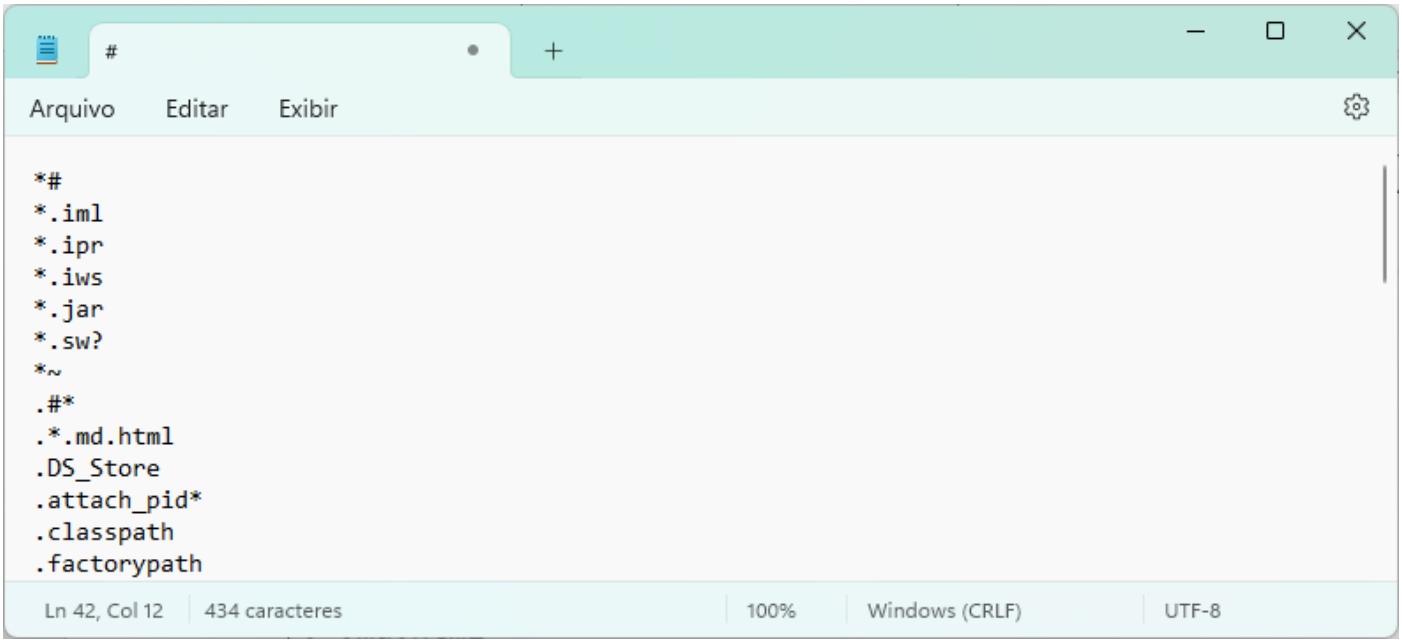
5 – Acesso o endereço https://github.com/spring-projects/spring-boot/blob/main/.gitignore



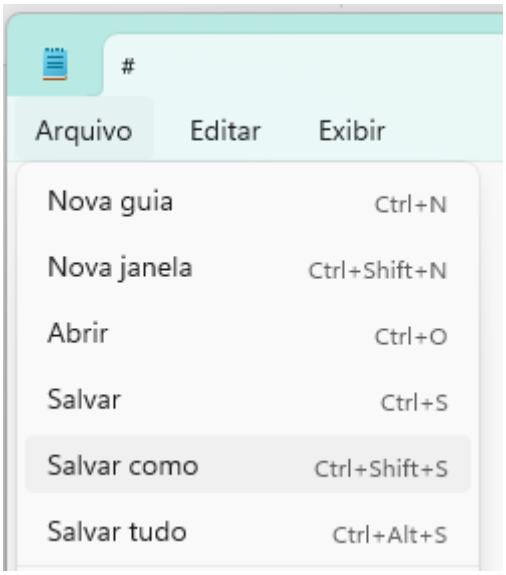
6 – Clique no botão copiar



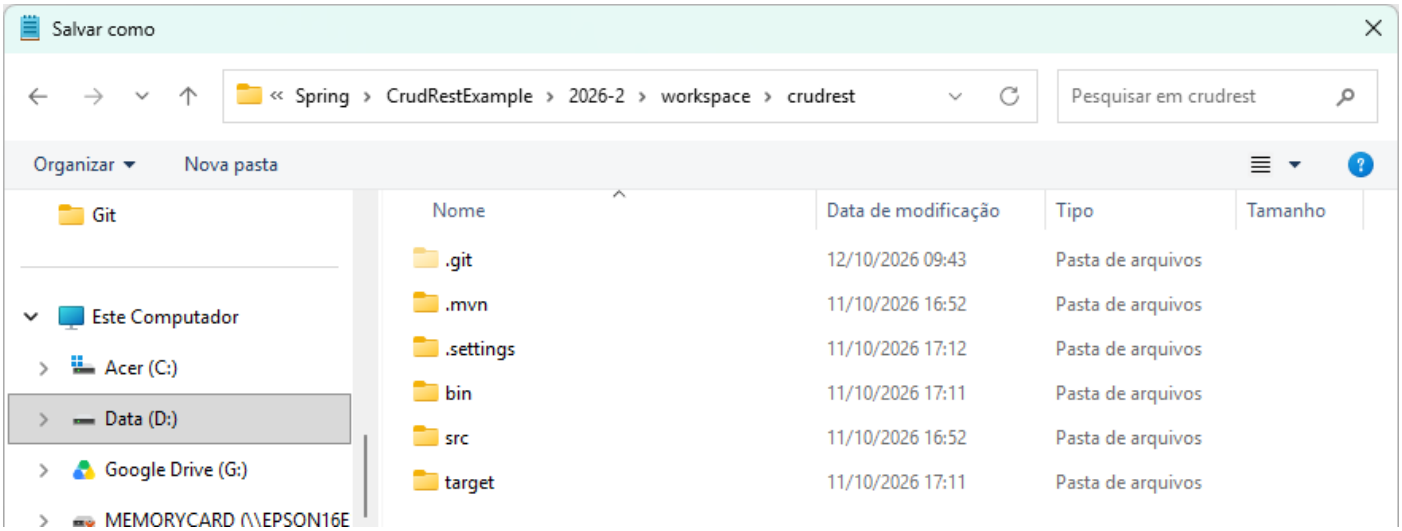
7 – Abra o notepad e cole o conteúdo copiado



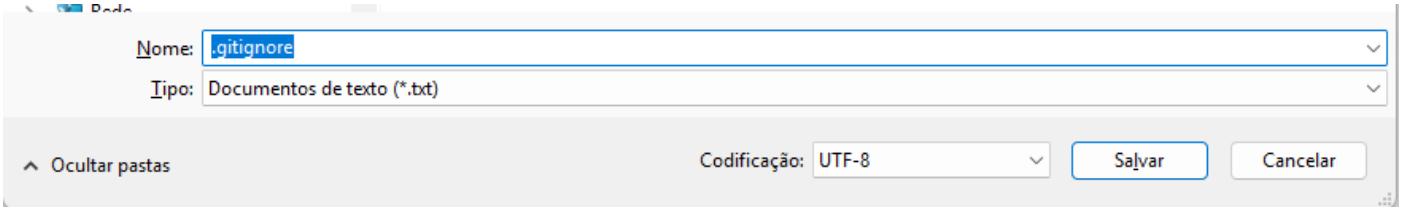
8 – Clique em Arquivo > Salvar como...



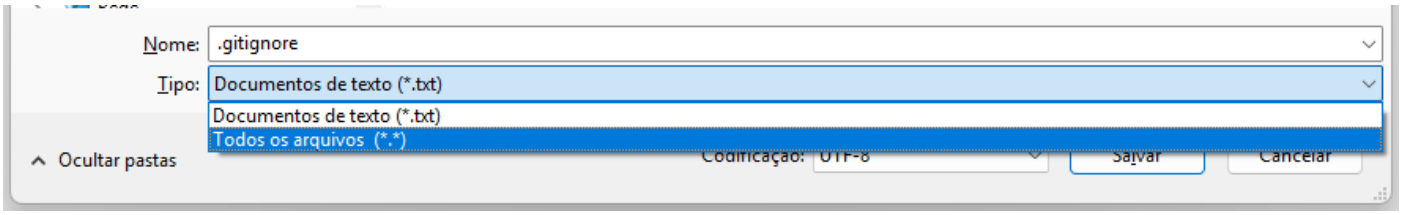
9 – Selecione a pasta do projeto (.../workspace/crudrest)



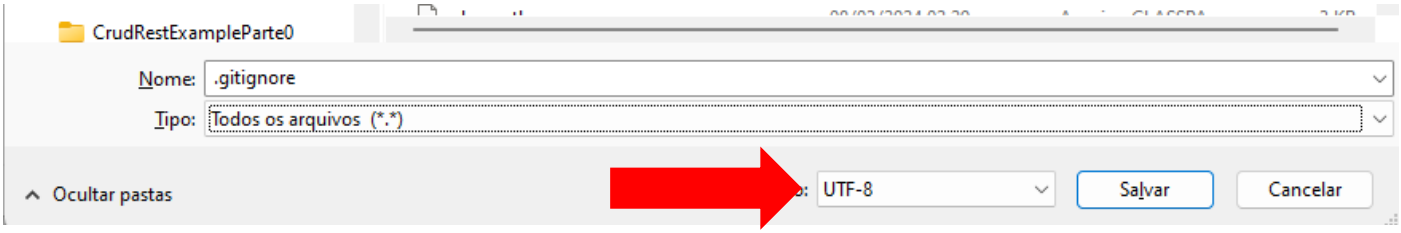
10 – Preencha o nomo do arquivo como “.gitignore” (observe o ponto inicial)



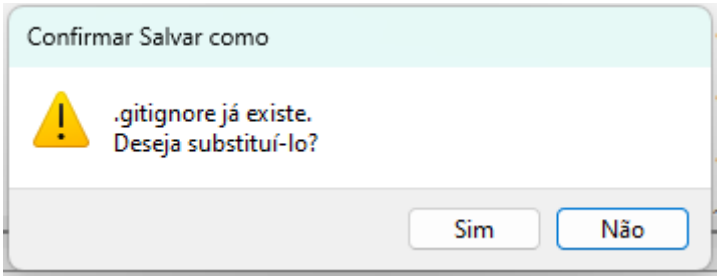
11 – Selecione o tipo “Todos os arquivos (*.*)”

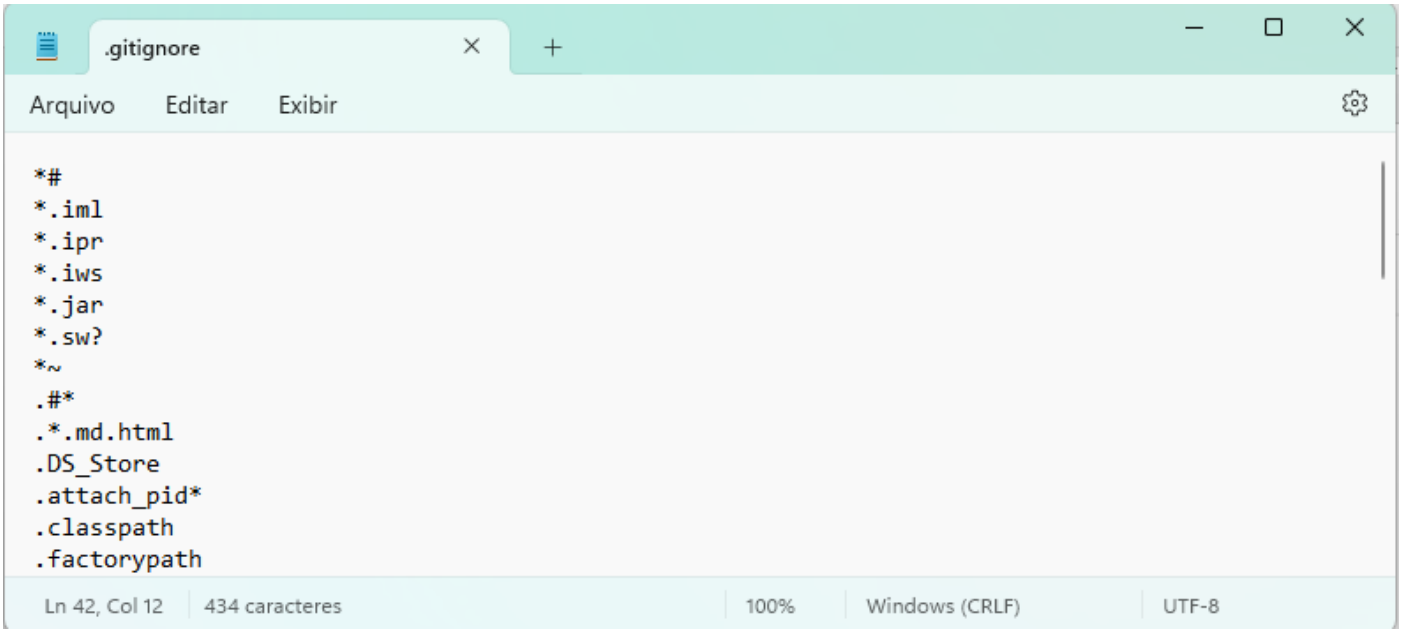


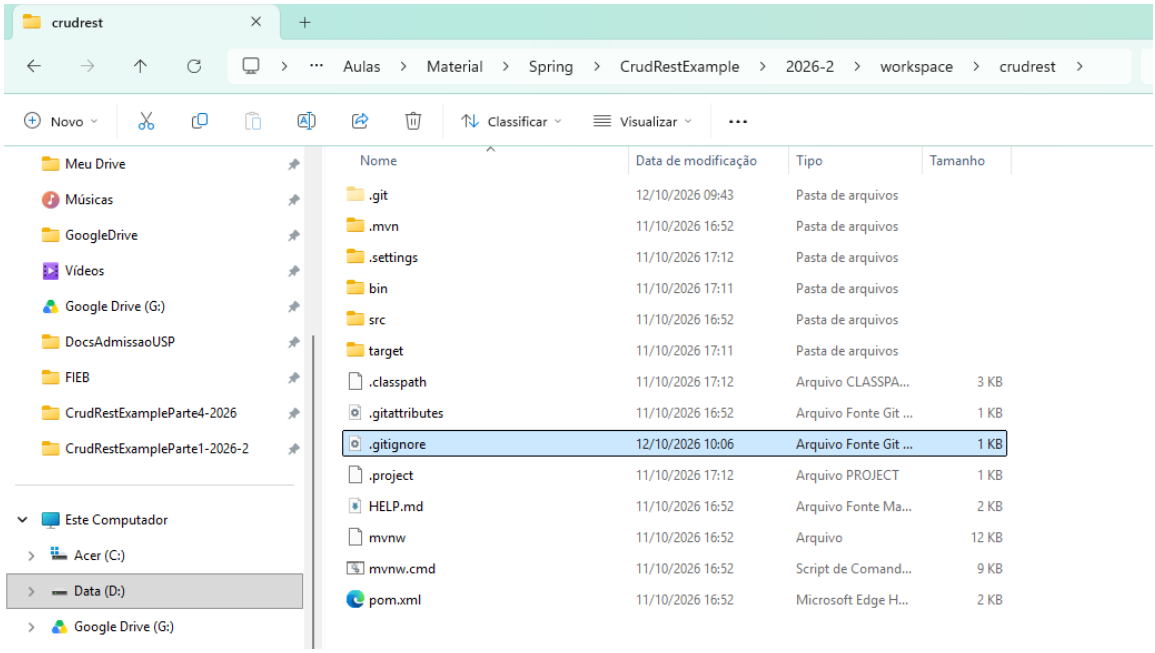
12 – Deixe a codificação como “UTF-8” e clique em Salvar



13 – Se o arquivo já existir e for perguntado se deseja substituí-lo, clique em Sim.







14 – Volte ao Git Bash e digite os comandos abaixo

```
git add .gitignore

git status
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git add .gitignore

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitattributes
        .mvn/
        HELP.md
        mvnw
        mvnw.cmd
        pom.xml
        src/
```

15 – Em seguida digite o comando

```
git commit -m "Add .gitignore"
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git commit -m "Add .gitignore"
[main (root-commit) 8c57988] Add .gitignore
 1 file changed, 44 insertions(+)
 create mode 100644 .gitignore

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Isso fará com que o git exclua os arquivos compilados deixando apenas o código fonte dentro do repositório e com isso deixando o repositório mais leve.

16 – Execute o comando abaixo para ver o status do “ambiente de trabalho” do git

```
git status
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitattributes
        .mvn/
        HELP.md
        mvnw
        mvnw.cmd
        pom.xml
        src/
```

17– Agora, vamos adicionar os fontes do projeto para “stage” do git, digite o comando abaixo

```
git add .
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git add .
warning: in the working copy of '.gitattributes', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.mvn/wrapper/maven-wrapper.properties', LF will be replaced by CRLF the next time Git tou
ches it
warning: in the working copy of 'HELP.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'mvnw.cmd', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/java/br/com/belval/crudrest/CrudrestApplication.java', LF will be replaced by CR
LF the next time Git touches it
warning: in the working copy of 'src/main/resources/application.properties', LF will be replaced by CRLF the next time Git
 touches it
warning: in the working copy of 'src/test/java/br/com/belval/crudrest/CrudrestApplicationTests.java', LF will be replaced
by CRLF the next time Git touches it

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

18 – Execute um comando “git status” para ver o que será adicionado ao repositório

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitattributes
        new file:   .mvn/wrapper/maven-wrapper.properties
        new file:   HELP.md
        new file:   mvnw
        new file:   mvnw.cmd
        new file:   pom.xml
        new file:   src/main/java/br/com/belval/crudrest/CrudrestApplication.java
        new file:   src/main/resources/application.properties
        new file:   src/test/java/br/com/belval/crudrest/CrudrestApplicationTests.java


takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

19 – Execute o comando abaixo para salvar a versão atual dos fontes no repositório.

```
git commit -m "Versão inicial do projeto crudrest"
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git commit -m "Versão inicial do projeto crudrest"
[main 4365911] Versão inicial do projeto crudrest
 9 files changed, 601 insertions(+)
 create mode 100644 .gitattributes
 create mode 100644 .mvn/wrapper/maven-wrapper.properties
 create mode 100644 HELP.md
 create mode 100644 mvnw
 create mode 100644 mvnw.cmd
 create mode 100644 pom.xml
 create mode 100644 src/main/java/br/com/belval/crudrest/CrudrestApplication.java
 create mode 100644 src/main/resources/application.properties
 create mode 100644 src/test/java/br/com/belval/crudrest/CrudrestApplicationTests.java

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

20 – Verifique o histórico das versões do seu software digitando o comando abaixo

```
git log
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git log
commit 43659111b29ec80764aa29a272f8c3bcc717f28a (HEAD -> main)
Author: ProfNpc <professornpc@gmail.com>
Date:   Mon Oct 12 10:14:09 2026 -0300

    Versão inicial do projeto crudrest

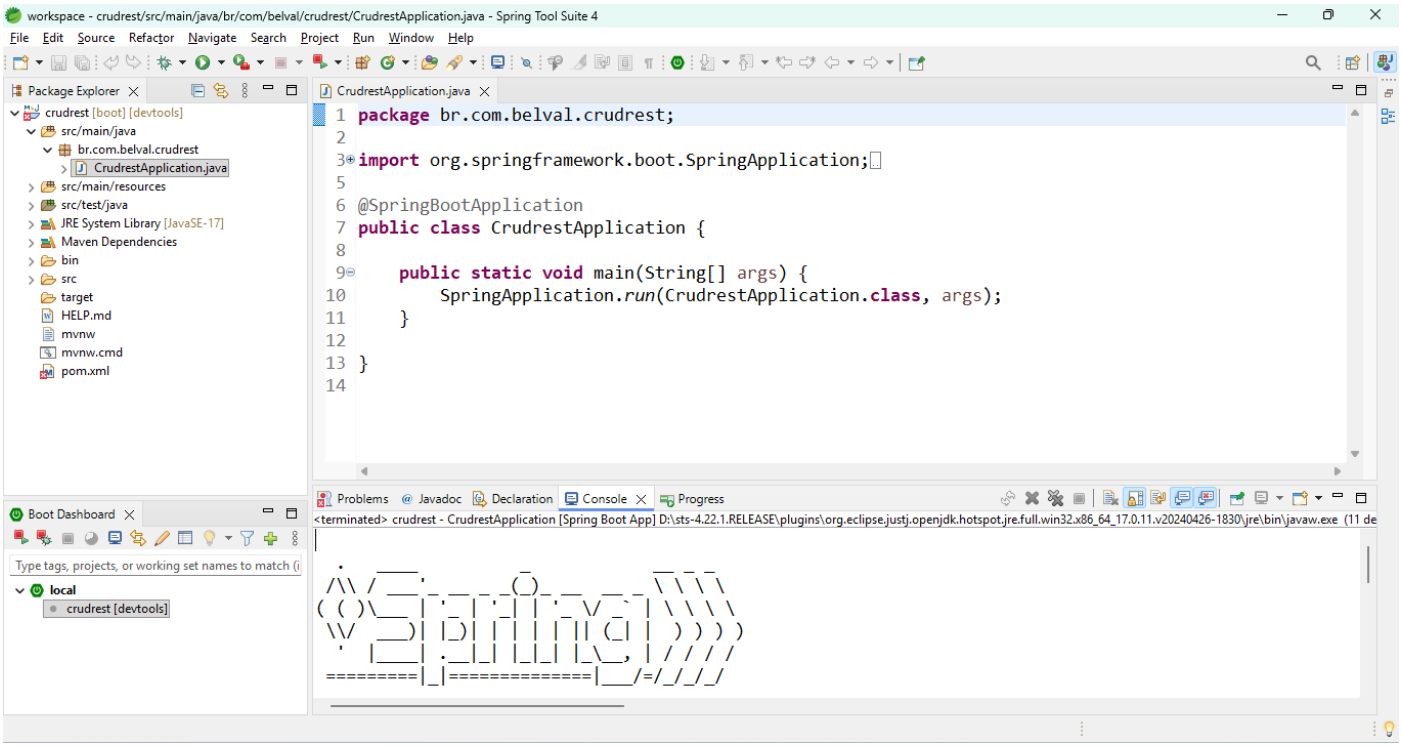
commit 8c579880fce208eeeb8b82f7a974203f3b293bb9
Author: ProfNpc <professornpc@gmail.com>
Date:   Mon Oct 12 10:09:57 2026 -0300

    Add .gitignore

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

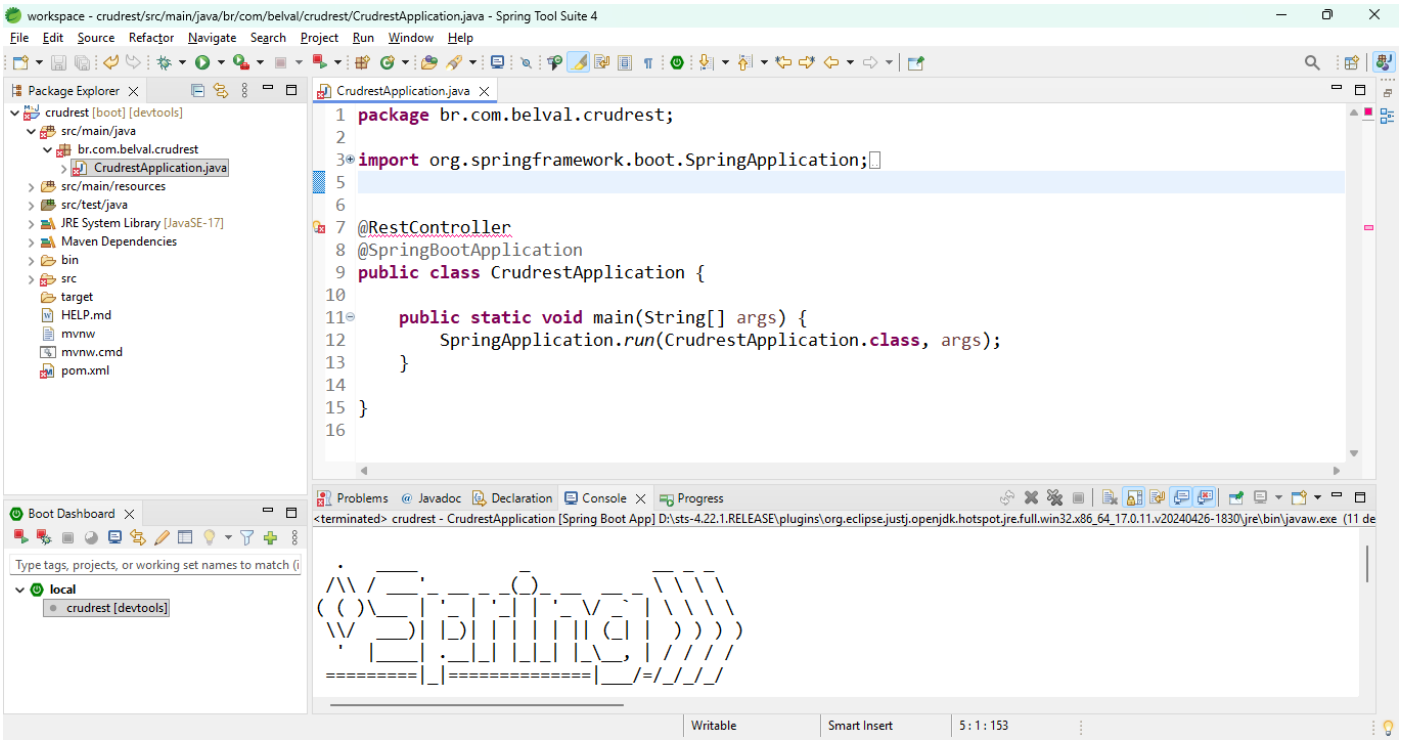
Fazendo a aplicação responder algo

1 – Abra o arquivo CrudrestApplication.java



2 – Para que nossa classe possa responder alguma coisa, a primeira coisa a fazer é “anotar” a classe como sendo um “Controller”. Talvez a versão mais simples de controller que podemos utilizar são os “RestControllers” que permitirão que nossos métodos respondam um texto diretamente para os clientes.

Coloque “@RestController” antes de “@SpringBootApplication”

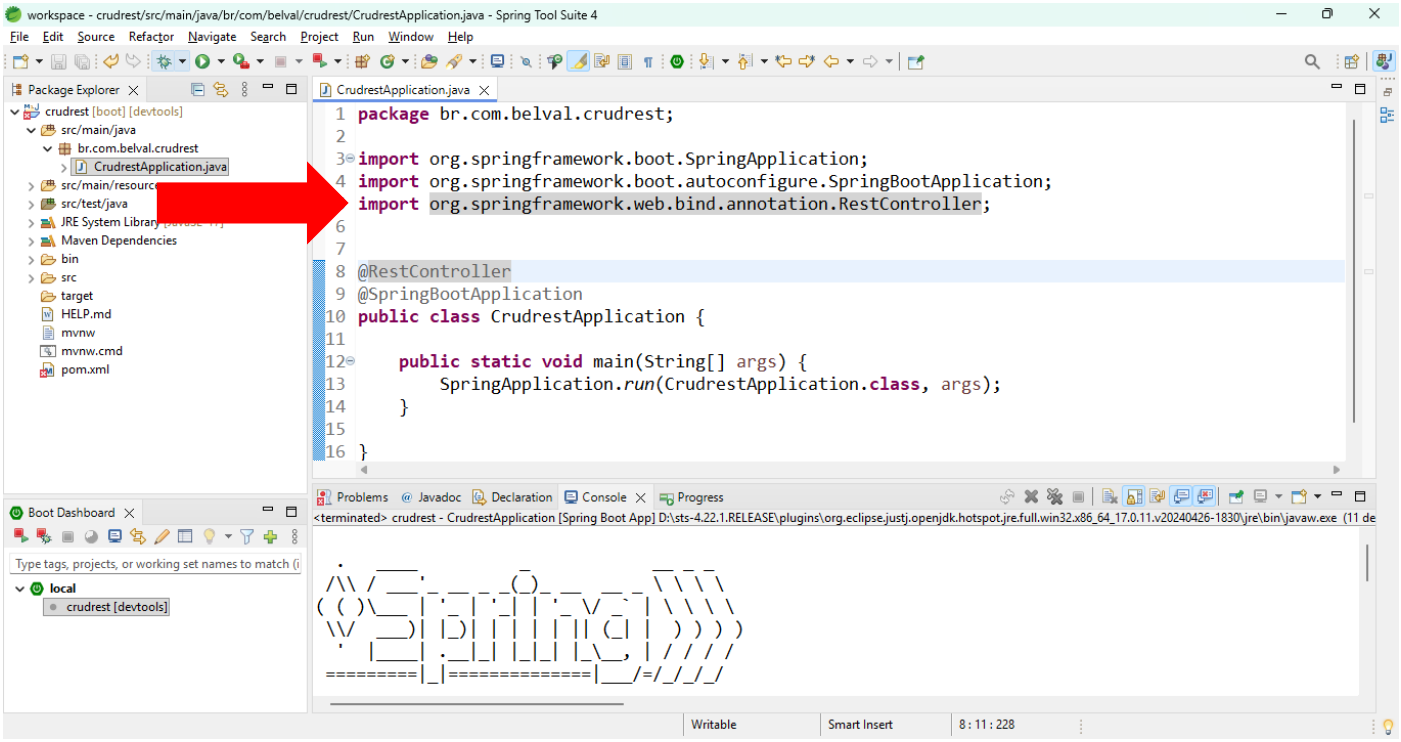


Como você pode ver, o nosso fonte apresenta um erro de compilação.

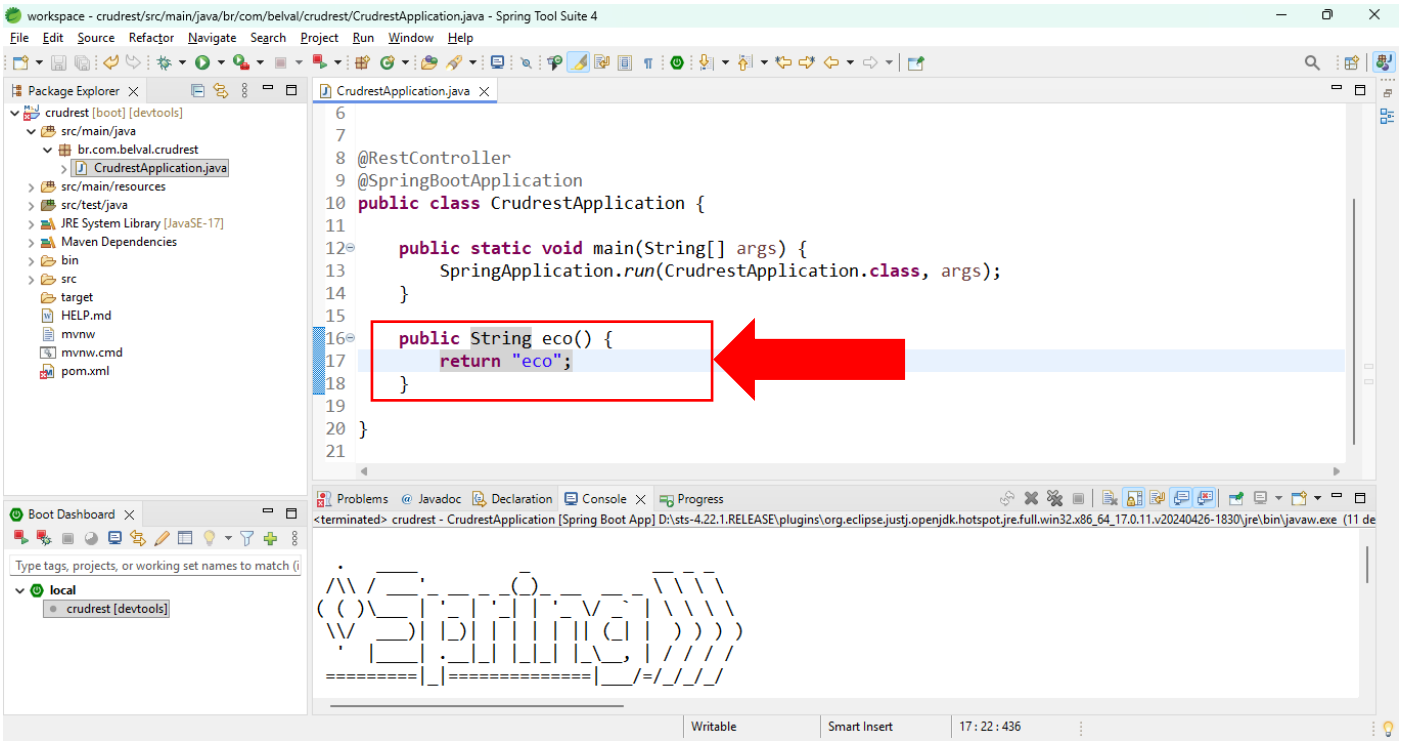
O problema aqui é que não adicionamos a importação da classe correspondente à anotação “@RestController”.

Para resolver isso podemos usar um atalho.

3 – Pressione “CTRL + SHIFT + O” que o ambiente automaticamente adicionará o *import* necessário.



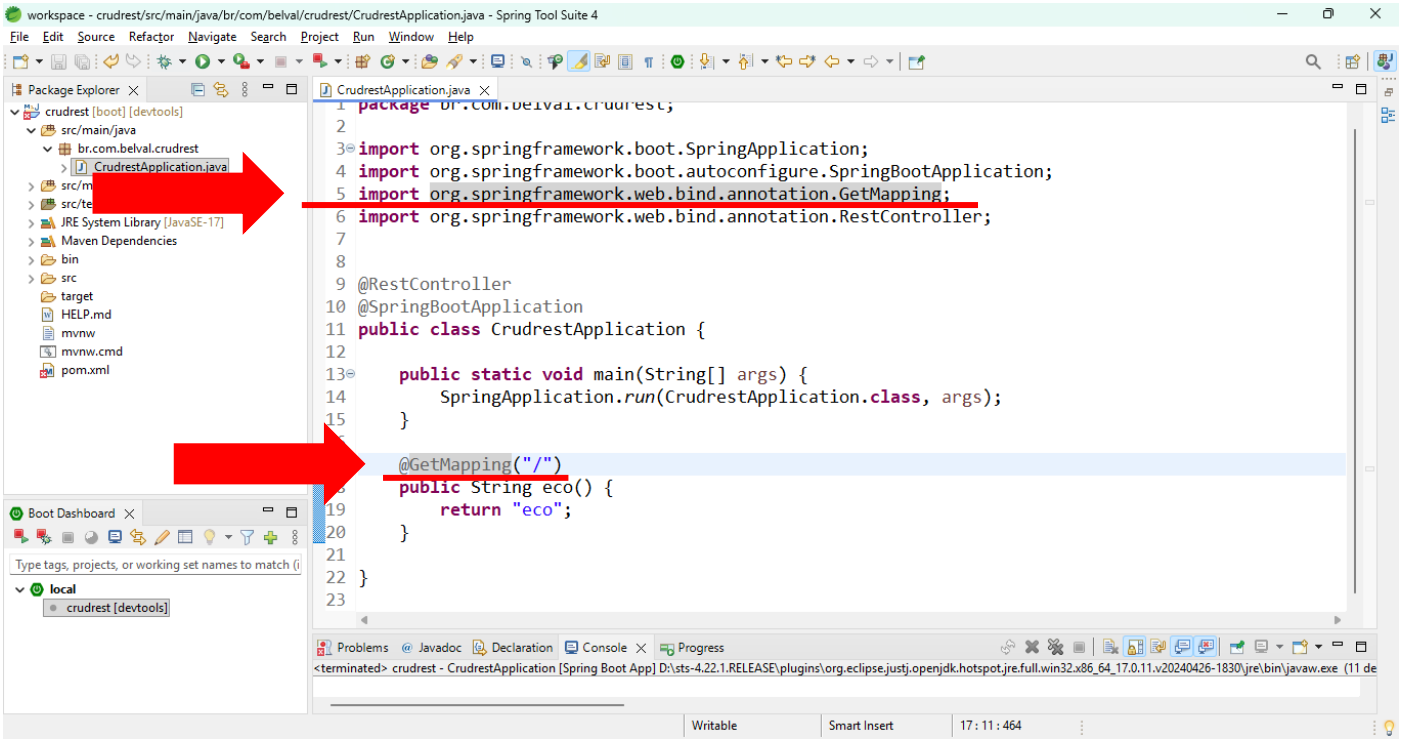
4 – Crie um método eco() como abaixo



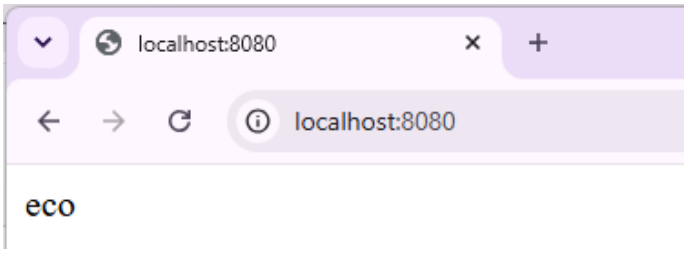
5 – Anote esse método com

```
@GetMapping(“/”)
```

E na sequência use o atalho “CTRL + SHIFT + O” para adicionar o *import* necessário.



6 – Execute a aplicação novamente e acesse o endereço “localhost:8080/”



7 – Pare a aplicação, e no git bash, execute o comando “git diff”

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git diff
warning: in the working copy of 'src/main/java/br/com/belval/crudrest/CrudrestApplication.java', LF will be replaced by CRLF the next
time Git touches it
diff --git a/src/main/java/br/com/belval/crudrest/CrudrestApplication.java b/src/main/java/br/com/belval/crudrest/CrudrestApplication.
java
index 28845bb..5171443 100644
--- a/src/main/java/br/com/belval/crudrest/CrudrestApplication.java
+++ b/src/main/java/br/com/belval/crudrest/CrudrestApplication.java
@@ -2,12 +2,21 @@ package br.com.belval.crudrest;

 import org.springframework.boot.SpringApplication;
 import org.springframework.boot.autoconfigure.SpringBootApplication;
+import org.springframework.web.bind.annotation.GetMapping;
+import org.springframework.web.bind.annotation.RestController;

+
+@RestController
 @SpringBootApplication
 public class CrudrestApplication {

        public static void main(String[] args) {
                SpringApplication.run(CrudrestApplication.class, args);
        }
+
+       @GetMapping("/")
+       public String eco() {
+               return "eco";
+       }

 }

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Observe que, as linhas que começam com “+” e que aqui também estão verdes, são aquelas que foram adicionadas em relação à última versão do nosso software.

8 – Se estiver tudo certo, execute os comandos abaixo e salve uma nova fotografia do nosso projeto.

```
git add .

git commit -m "Add método eco()"
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git add .
warning: in the working copy of 'src/main/java/br/com/belval/crudrest/CrudrestApplication.java', LF will be replaced by CRLF the next
time Git touches it

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git commit -m "Add método eco()"
[main 3dbbfb6] Add método eco()
 1 file changed, 9 insertions(+)

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

9 – Para verificar o histórico do nosso projeto, execute o comando “git log”

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git log
commit 3dbbfb60801181f09134514fad8bdbfe670d204e (HEAD -> main)
Author: ProfNpc <professornpc@gmail.com>
Date:   Mon Oct 12 10:23:16 2026 -0300

    Add método eco()

commit 43659111b29ec80764aa29a272f8c3bcc717f28a
Author: ProfNpc <professornpc@gmail.com>
Date:   Mon Oct 12 10:14:09 2026 -0300

    Versão inicial do projeto crudrest

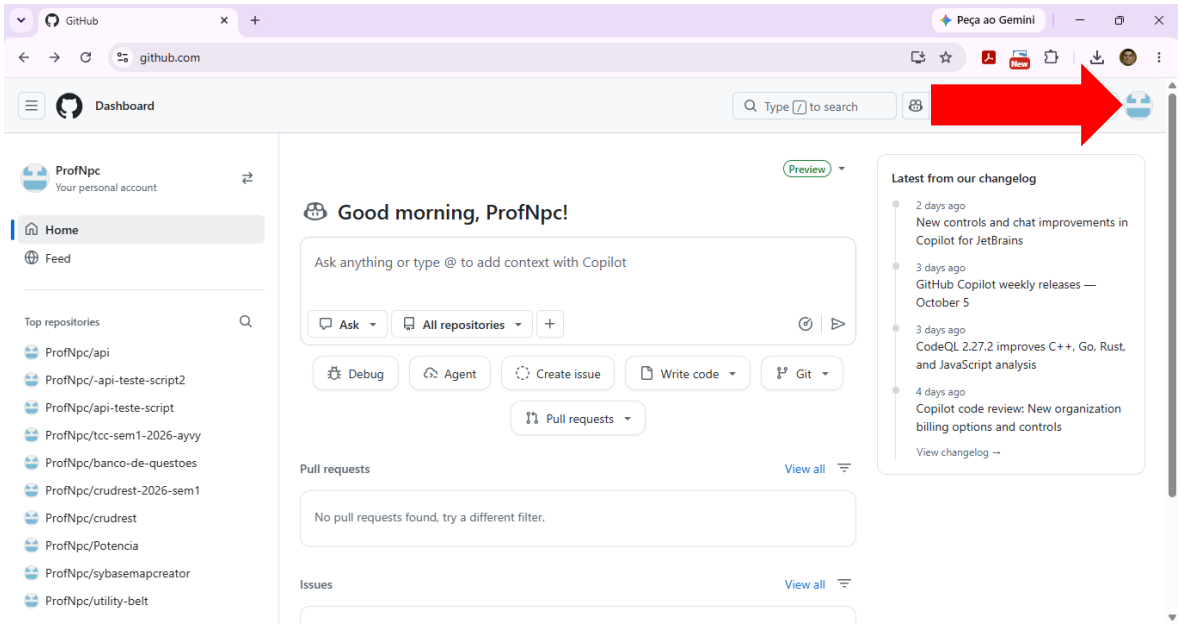
commit 8c579880fce208eeeb8b82f7a974203f3b293bb9
Author: ProfNpc <professornpc@gmail.com>
Date:   Mon Oct 12 10:09:57 2026 -0300

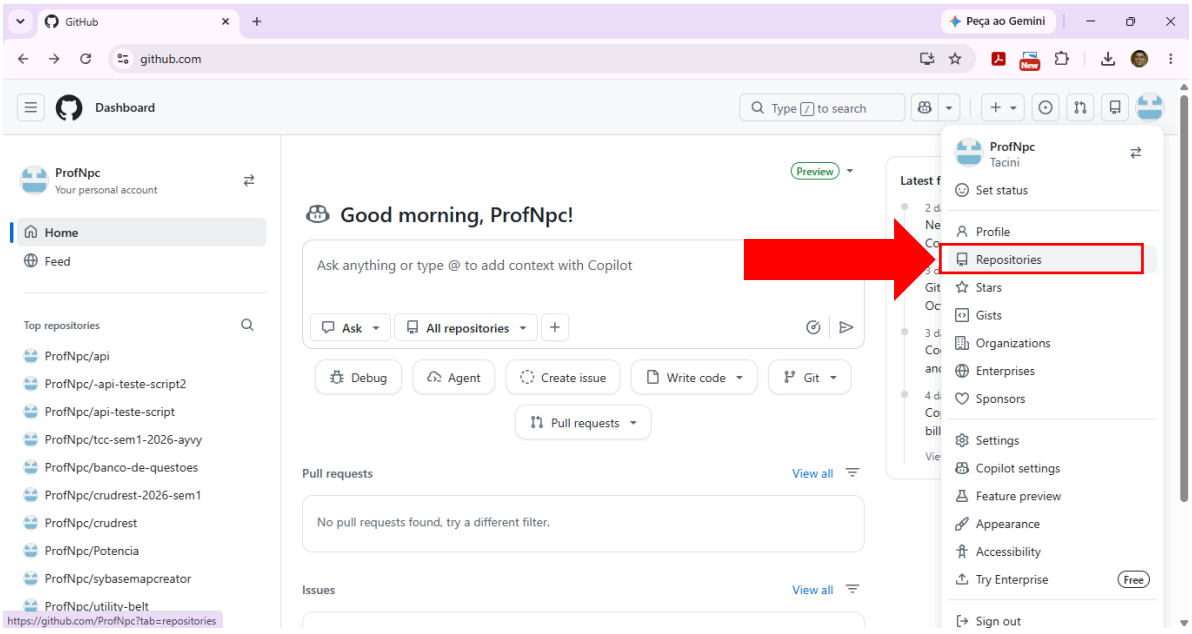
    Add .gitignore

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

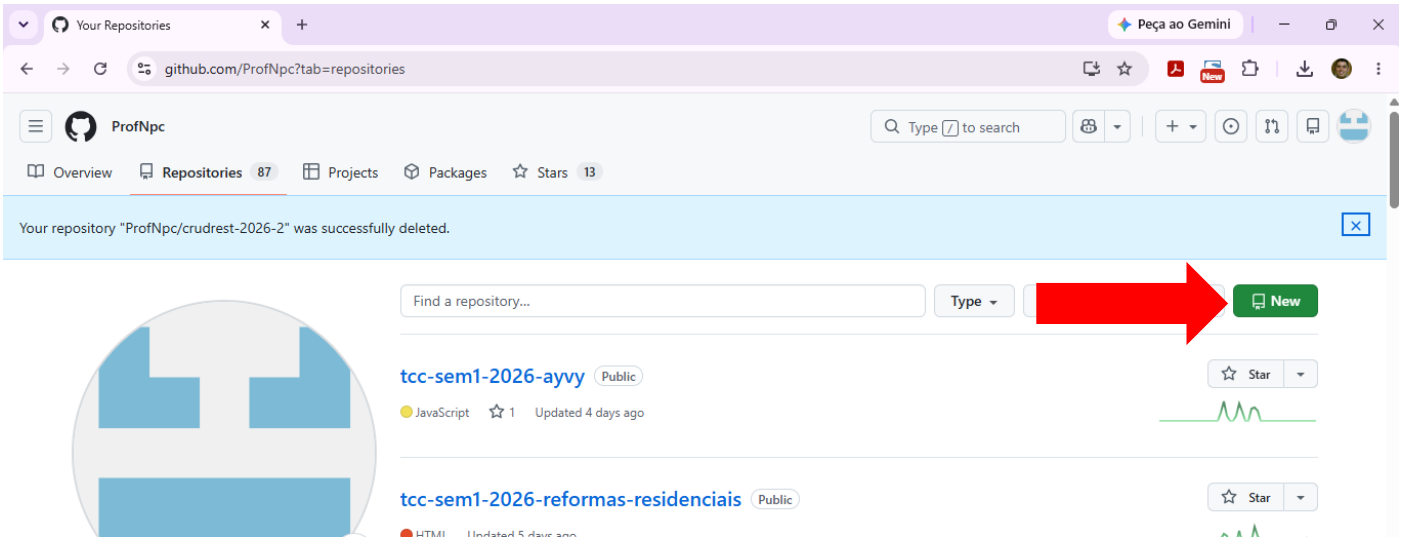
Criando um repositório no GitHub

1 – Acesse sua conta no GitHub e clique na imagem do seu perfil no canto superior direito, depois clique em “Your repositories”.

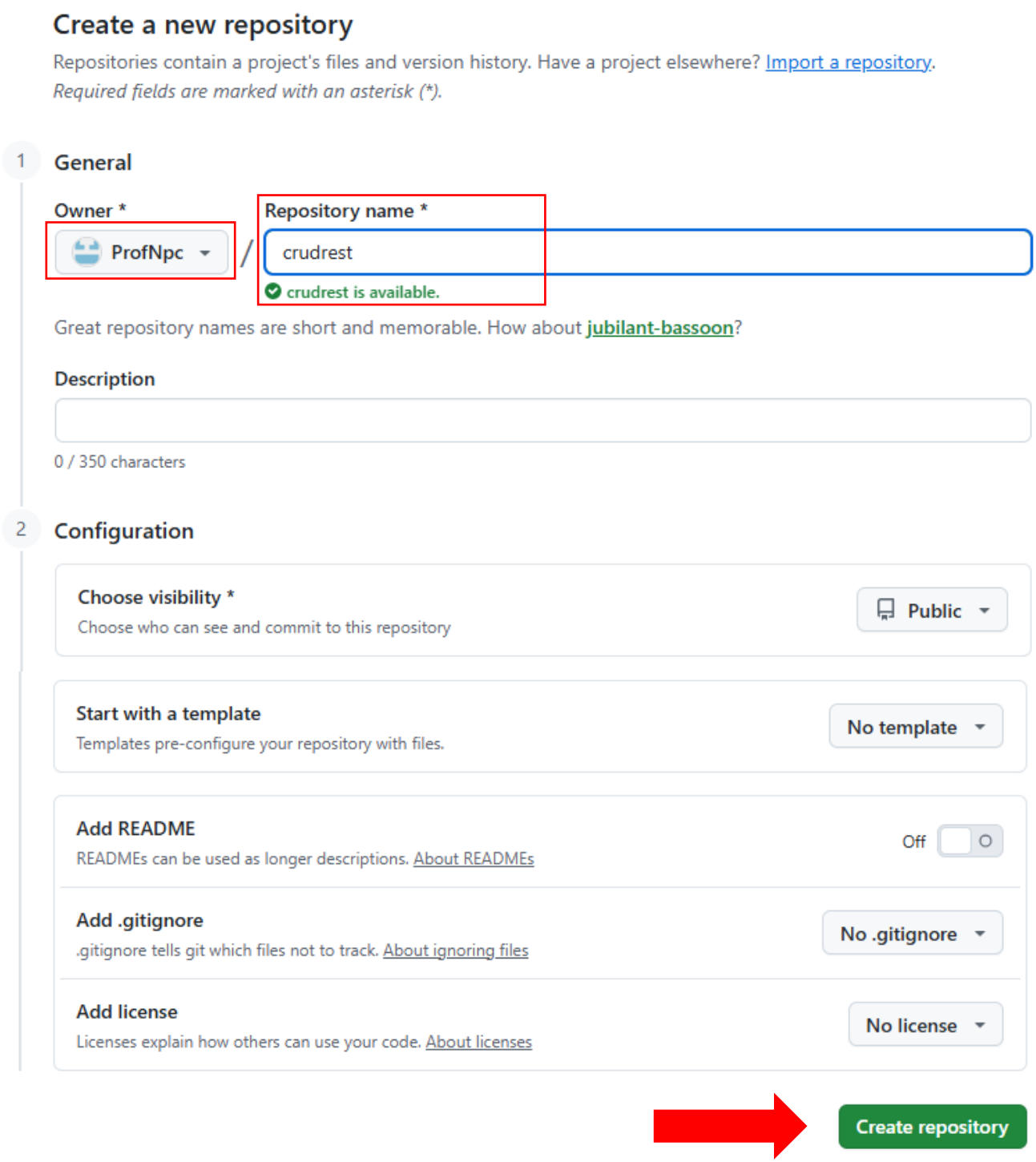


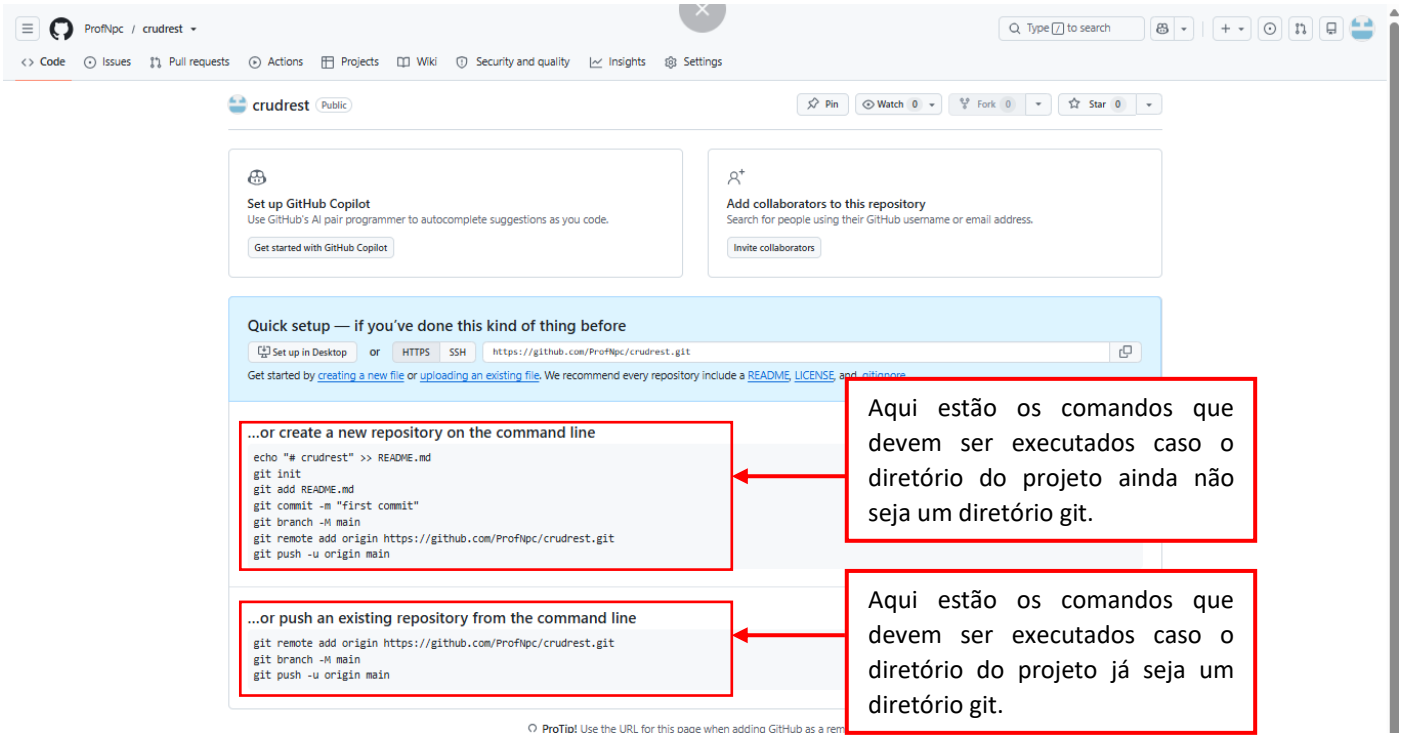


2 – Clique em “New” para criar um novo repositório.



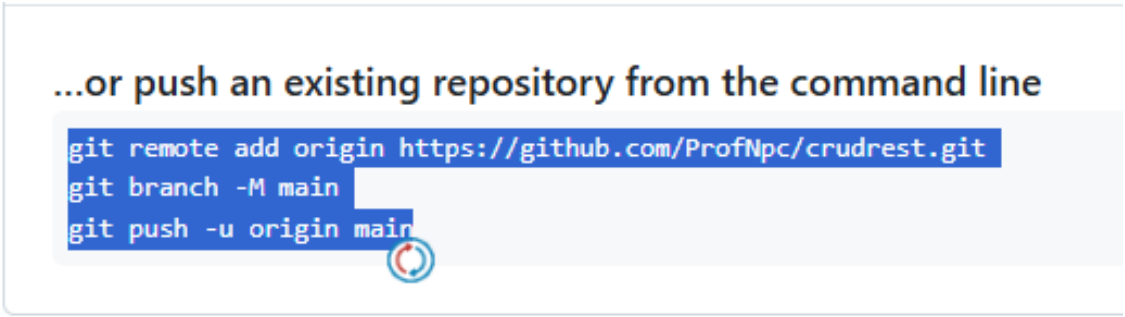
3 – Preencha o campo “Owner”, o nome do repositório e clique em “Create repository”.





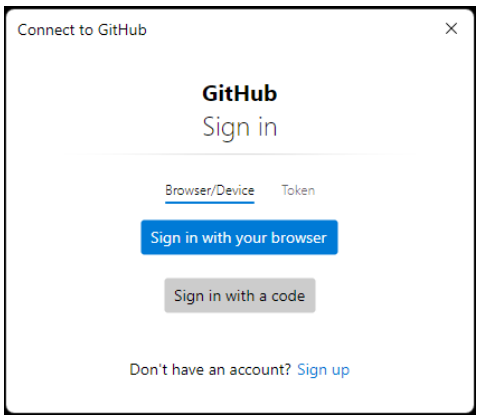
Como nós já configuramos o diretório do nosso projeto como um diretório gerenciado pelo Git, vamos utilizar o segundo grupo de comandos para enviar nosso projeto para o GitHub.

4 – Copie o segundo grupo de comandos da página que abriu pra você quando você criou o novo repositório "crudrest" e execute-os no GitBash. **Atenção: não copie as instruções desse documento porque essas instruções se aplicam ao repositório do usuário ProfNpc e não ao seu.**

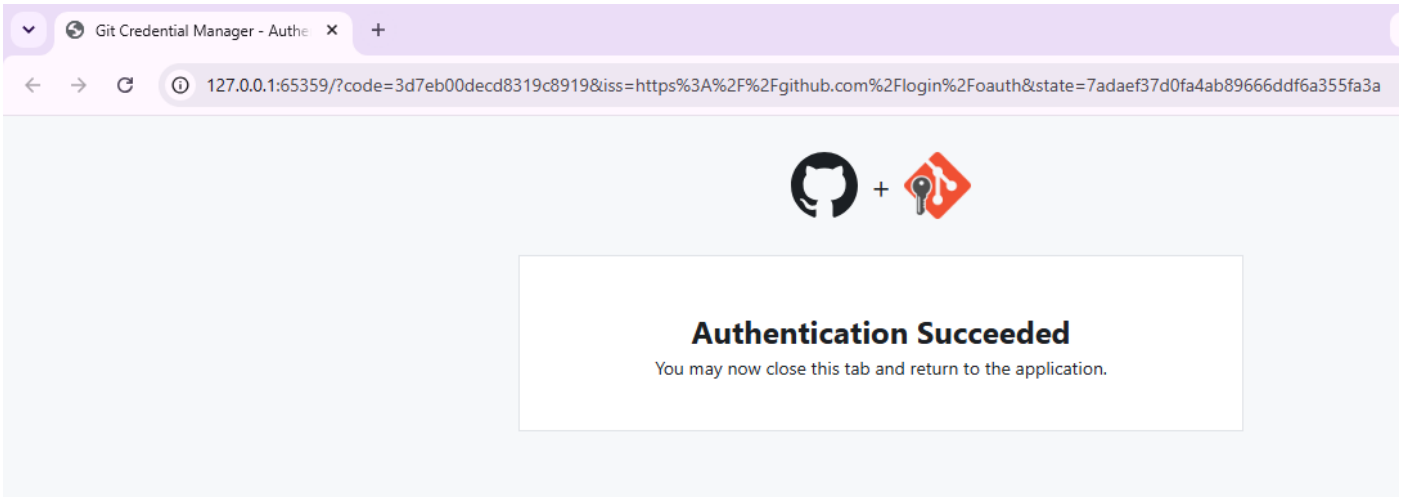



```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git remote add origin https://github.com/ProfNpc/crudrest.git
git branch -M main
git push -u origin main
```

5 – Deve aparecer a seguinte janela, clique em "Sign in with your browser"



Se você não estiver logado no GitHub precisará fazê-lo, após isso, deve aparecer a seguinte página

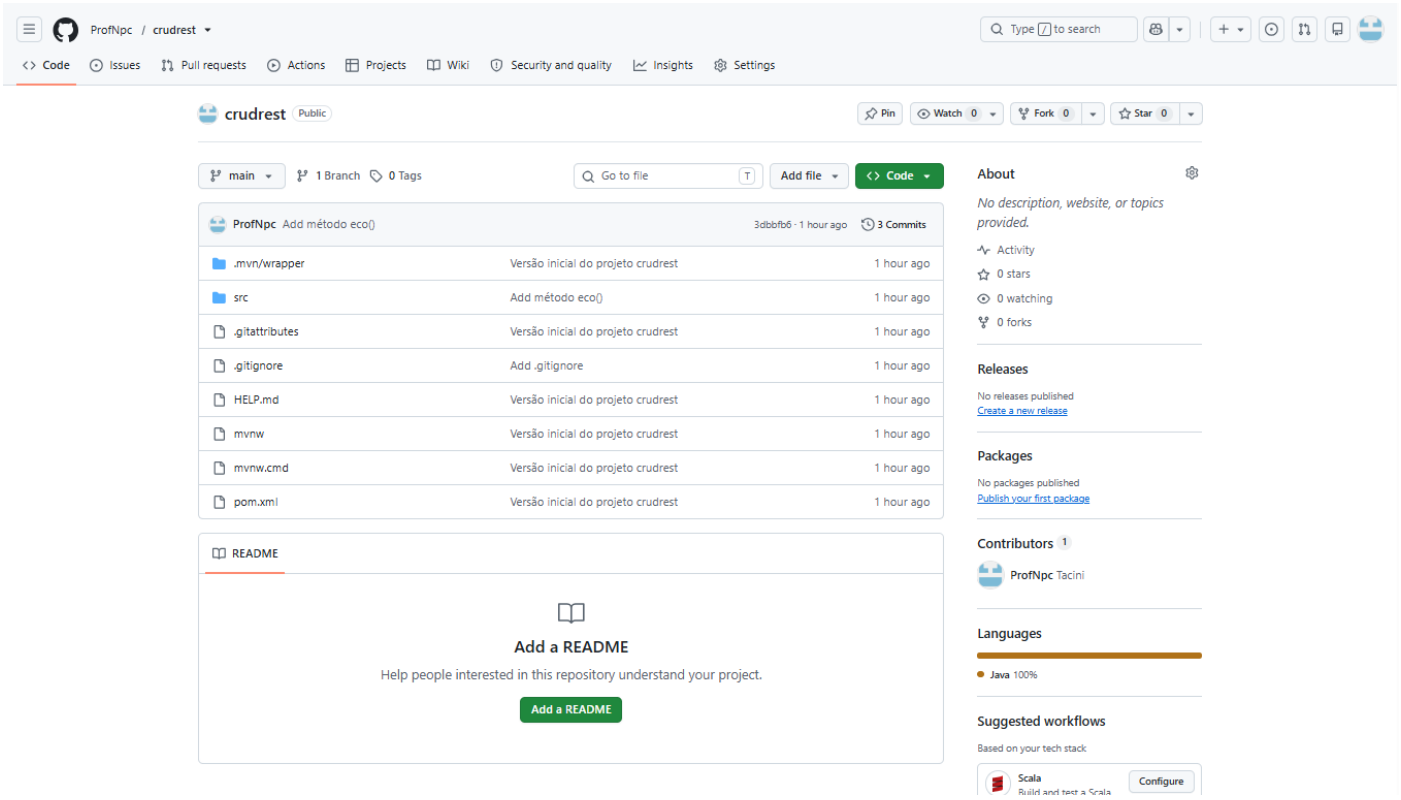


No GitBash, deverá aparecer a seguinte mensagem mostrando que o repositório local com seu histórico de commit's foi enviado para o GitHub

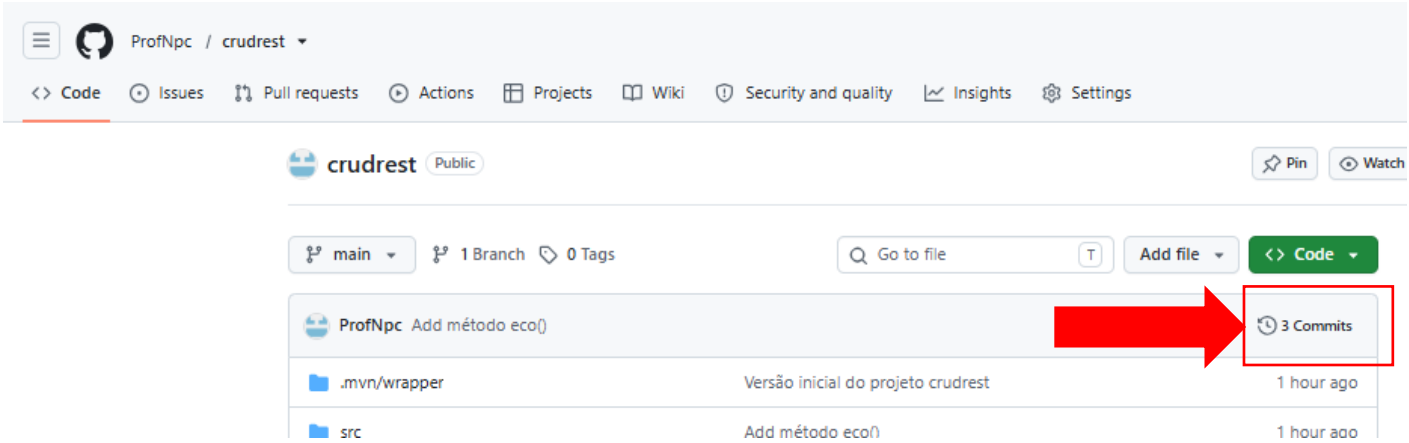
```
info: please complete authentication in your browser...
Enumerating objects: 40, done.
Counting objects: 100% (40/40), done.
Delta compression using up to 8 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (40/40), 10.86 KiB | 370.00 KiB/s, done.
Total 40 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/ProfNpc/crudrest.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

Volte na página do seu repositório e dê um *refresh* com F5 e verá que a página agora apresenta o mesmo conteúdo do repositório local.



Se clicar na indicação do número de *commit's*



Poderá ver o mesmo que mostra o comando git log

